

Hybrid Computing Technology

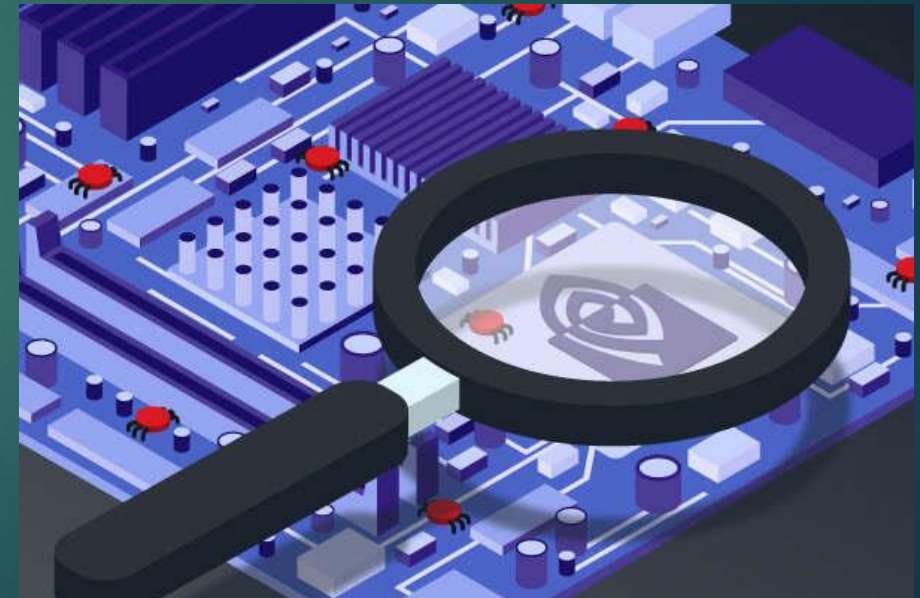
1

CUDA Programming

ผศ. อัครินทร์ คุณกิตติ

ASST. PROF. AKHARIN KHUNKITTI

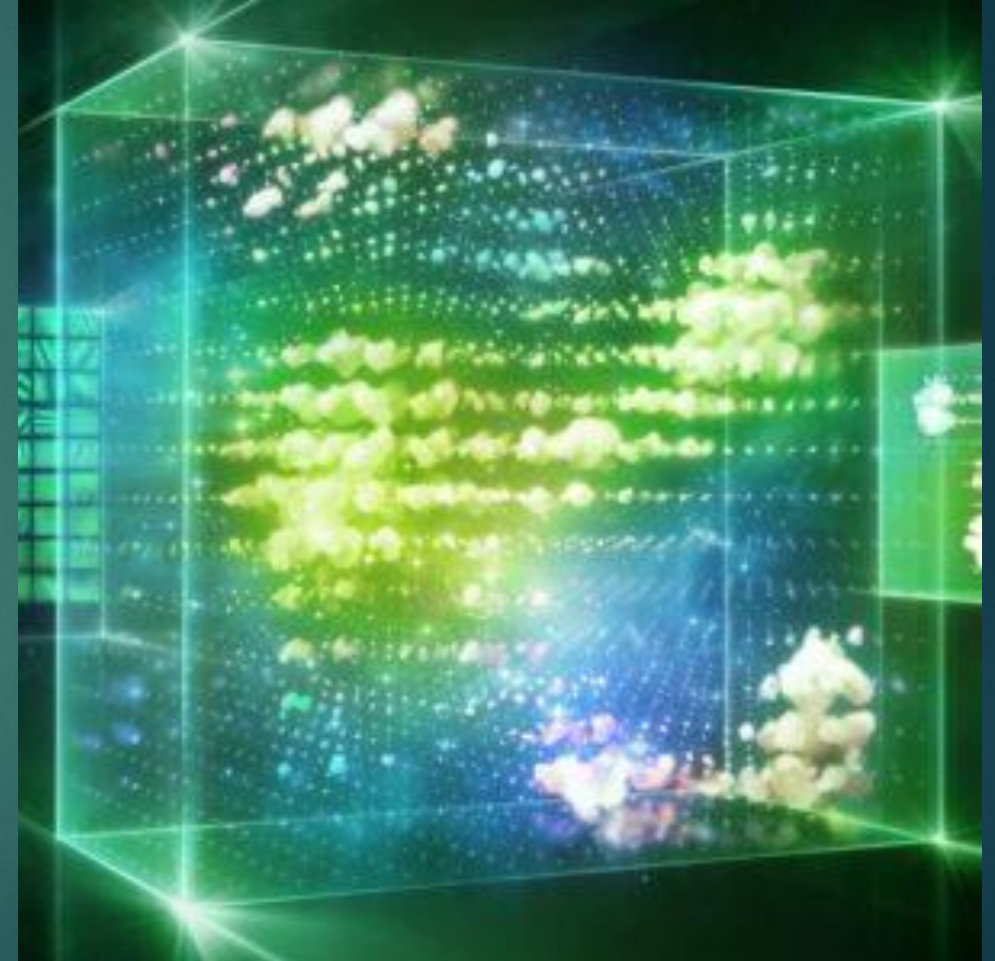
FACULTY OF INFORMATION TECHNOLOGY
KMITL



Topics

2

- ▶ CUDA Introduction
- ▶ CUDA Programming Model
- ▶ CUDA Memory Model
- ▶ CUDA APIs
- ▶ Example Program
- ▶ Pro & Con
- ▶ Conclusion



CUDA Introduction

3

What is CUDA?

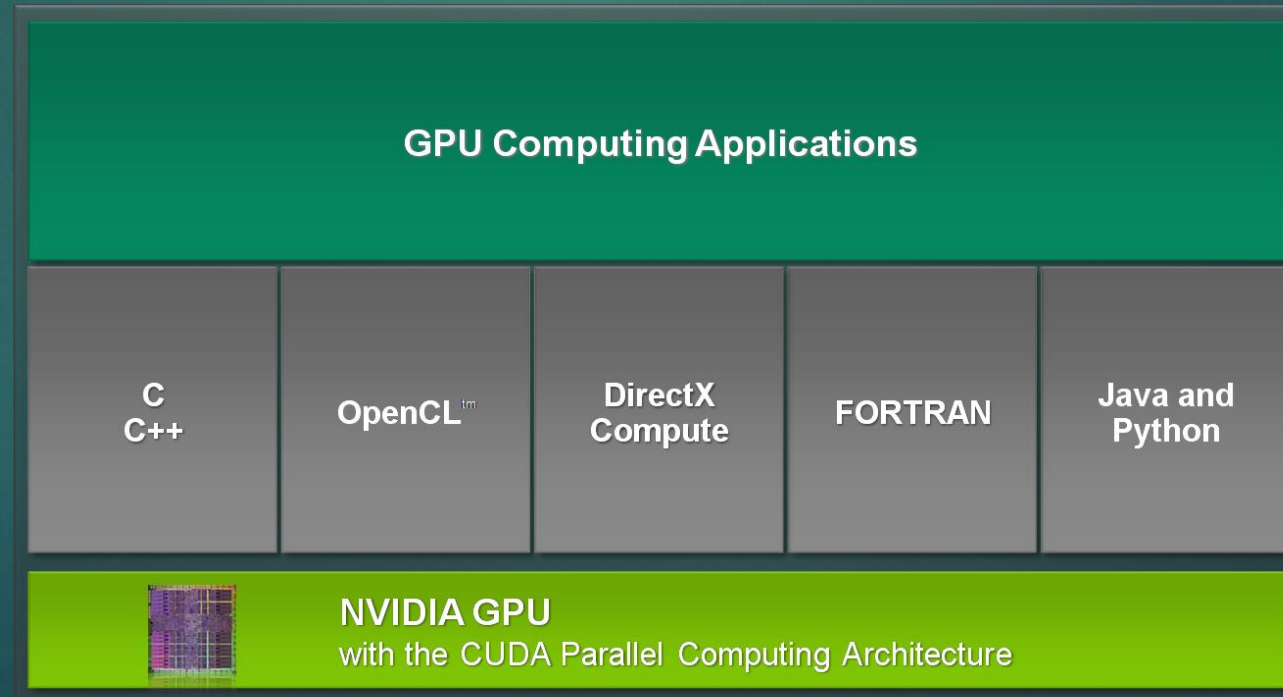
- **Compute Unified Device Architecture.**
- A powerful parallel programming model for issuing and managing computations on the GPU without mapping them to a graphics API.
- Heterogenous - mixed serial-parallel programming
- Scalable - hierarchical thread execution model
- Accessible - minimal but expressive changes to C

CUDA Introduction

4

CUDA is NVIDIA's general purpose parallel computing architecture .

- designed for calculation-intensive computation on GPU hardware
- CUDA is not a language, it is an API
- we will mostly concentrate on the C implementation of CUDA



What is GPGPU ?

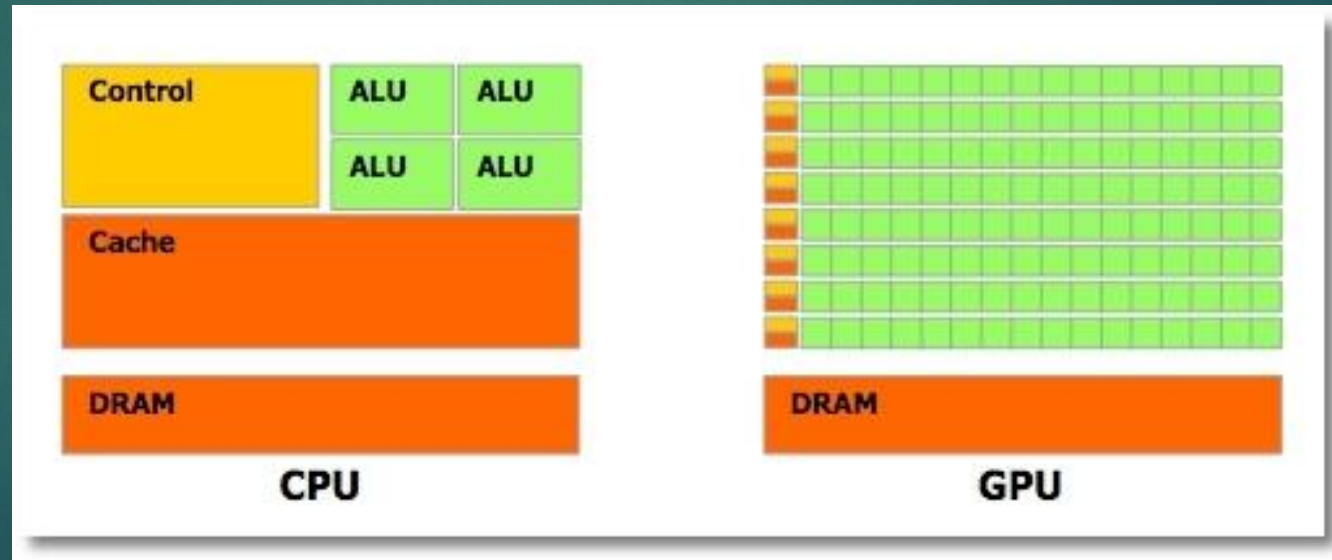


- ▶ General Purpose computation using GPU in applications other than 3D graphics
 - ▶ GPU accelerates critical path of application
- ▶ Data parallel algorithms leverage GPU attributes
 - ▶ Large data arrays, streaming throughput
 - ▶ Fine-grain SIMD parallelism
 - ▶ Low-latency floating point (FP) computation
- ▶ Applications – see [//GPGPU.org](http://GPGPU.org)
 - ▶ Game effects (FX) physics, image processing
 - ▶ Physical modeling, computational engineering, matrix algebra, convolution, correlation, sorting

CUDA Goals

6

- ▶ Scale code to hundreds of cores running thousands of threads
- ▶ The task runs on the gpu independently from the cpu

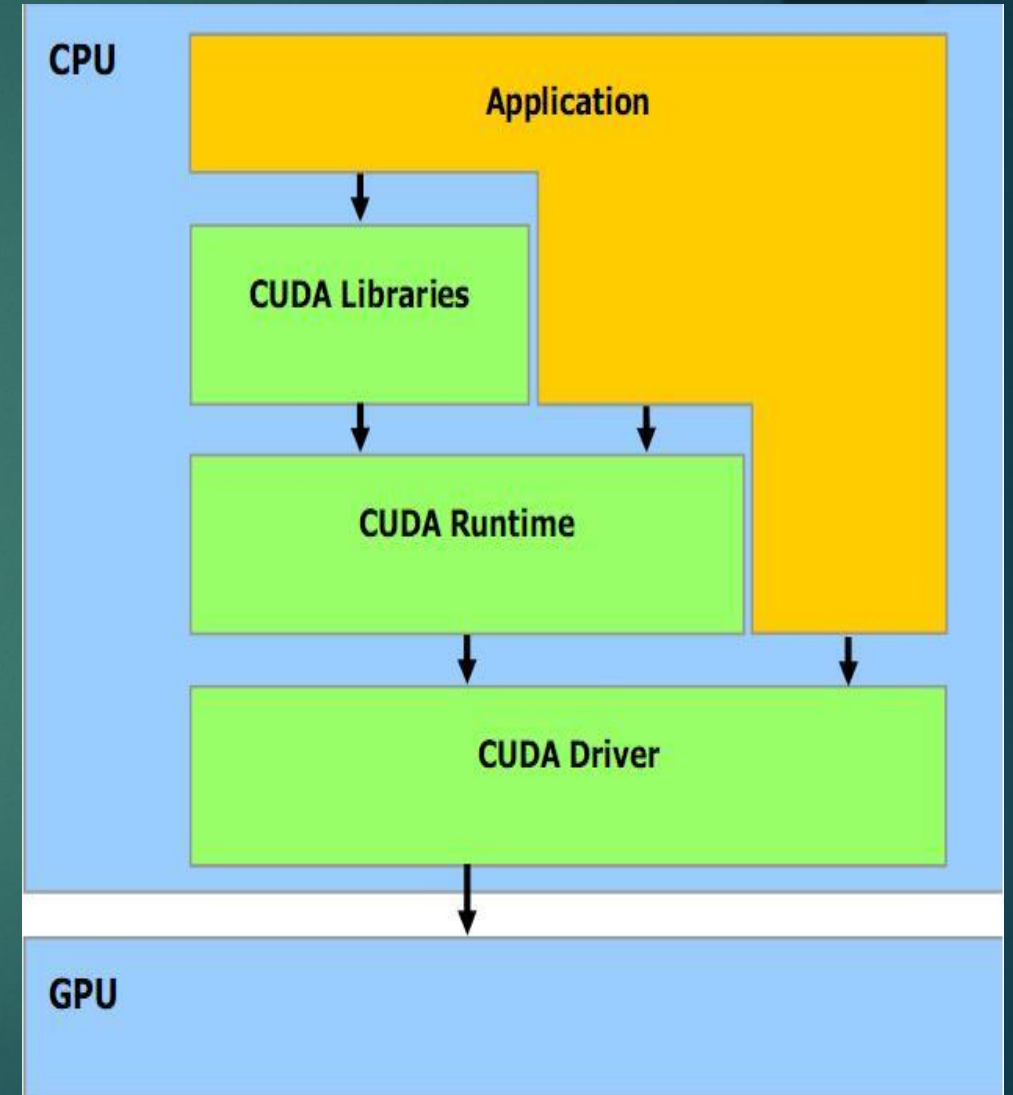


CUDA Introduction

7

Software Stack:

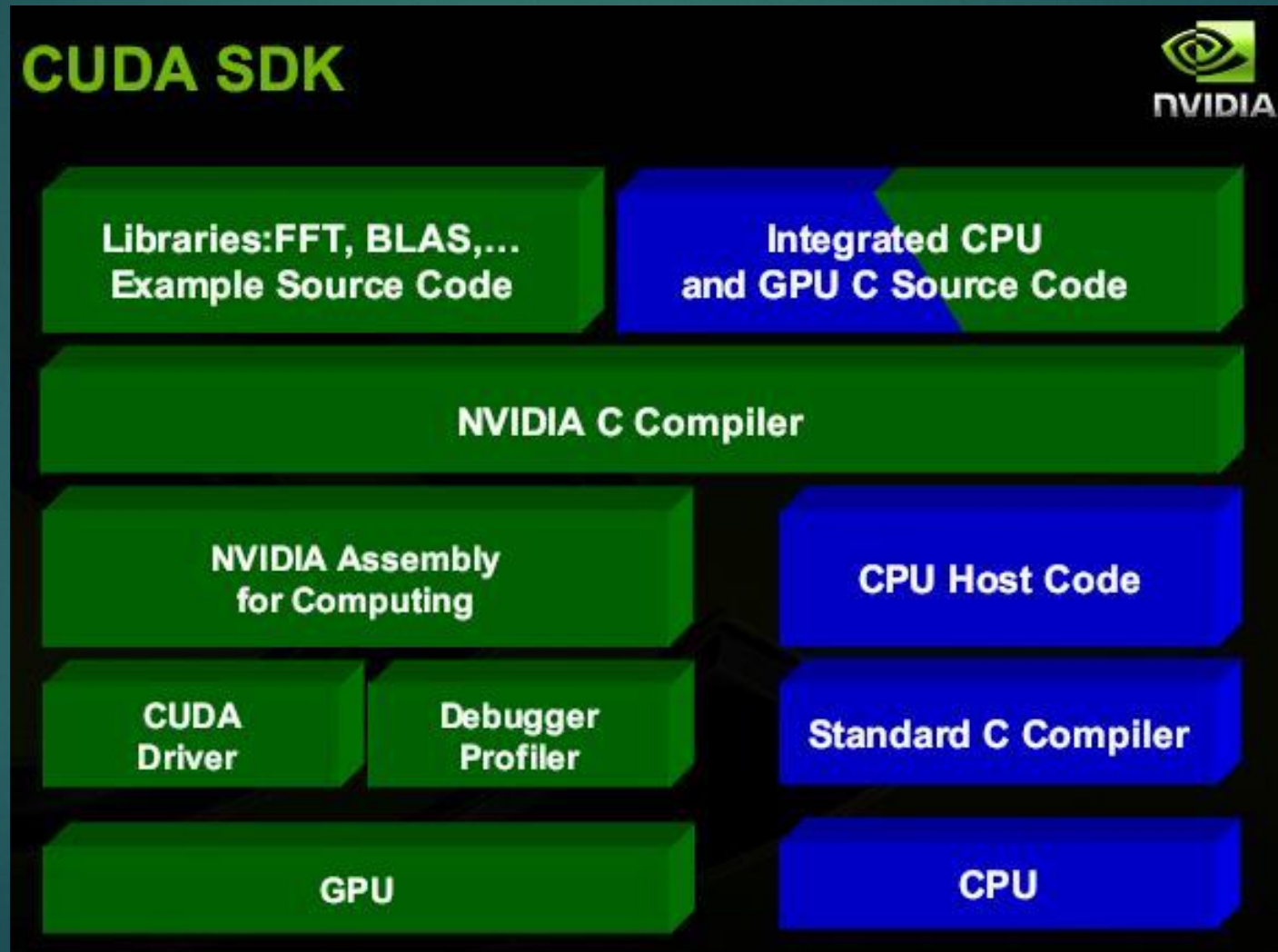
- Libraries:
CUFFT & CUBLAS
- Runtime:
Common component
Device component
Host component
- Driver:
Driver API



CUDA Introduction

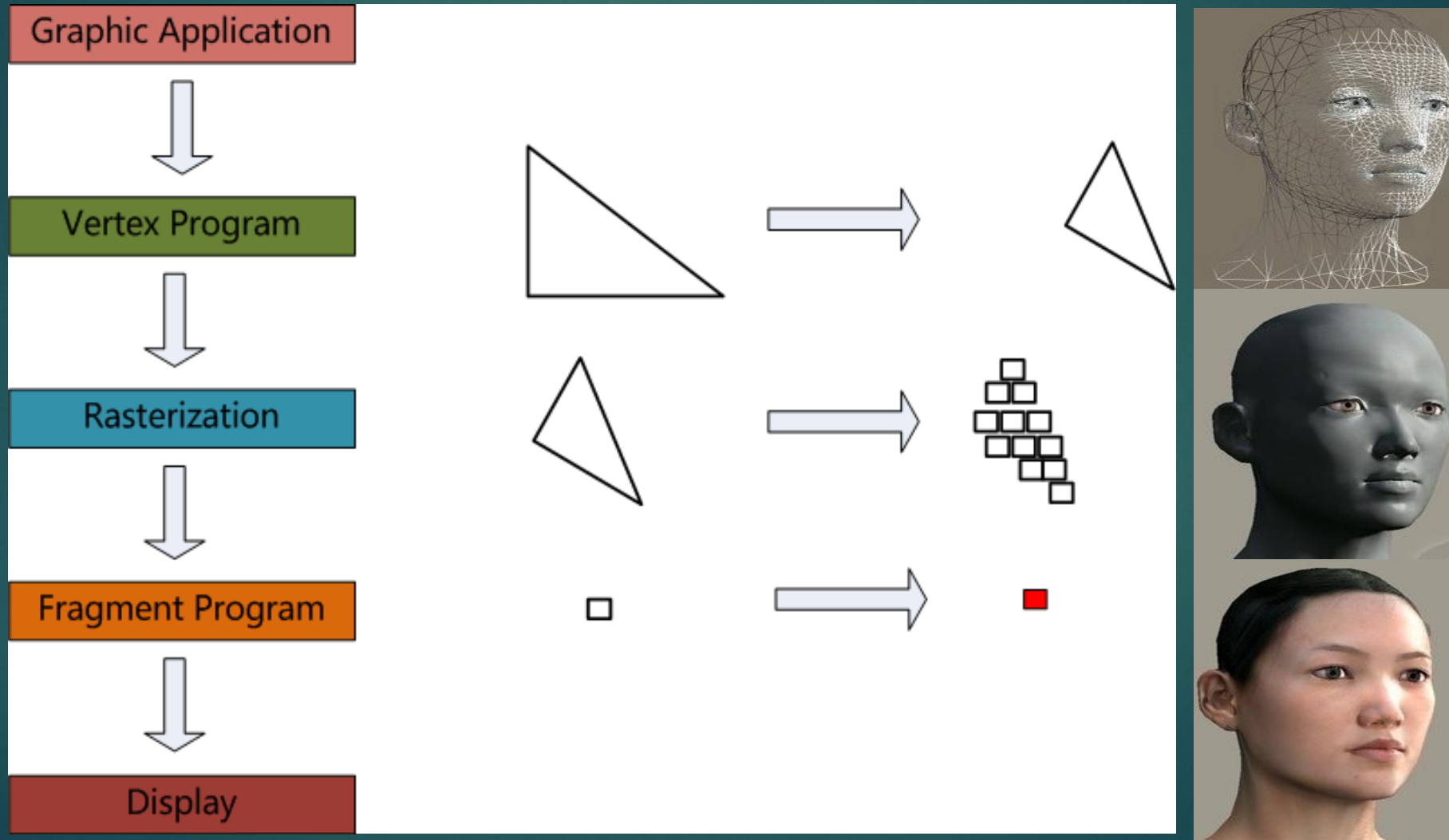
8

CUDA SDK



Motivation

GPU Programming Model for Graphics



Motivation

10

GPGPU Programming Model

Trick the GPU into general-purpose computing by casting problem as graphics

- Turn data into images ("texture maps")
- Turn algorithms into image synthesis ("rendering passes")

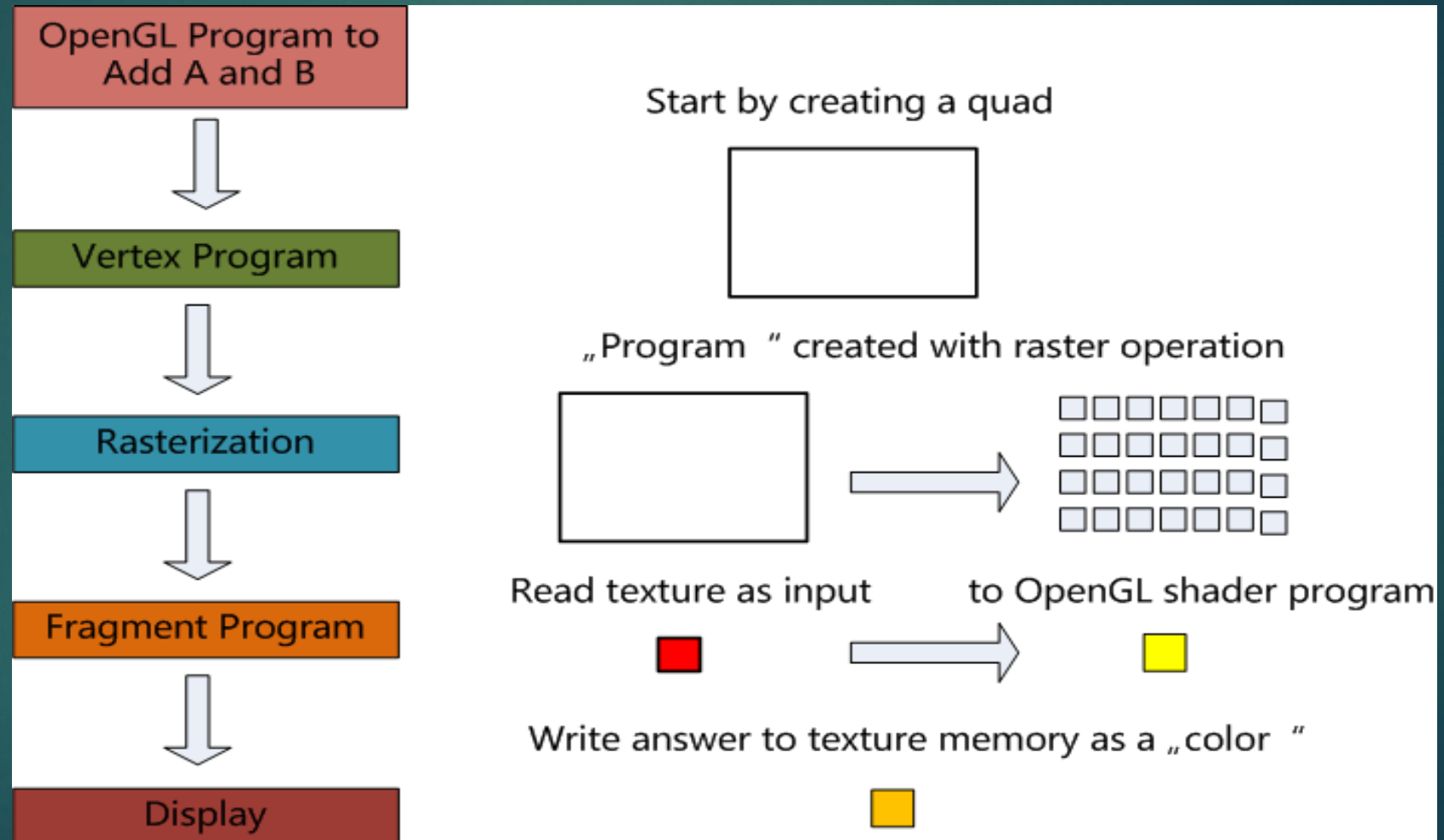
Drawback:

- Tough learning curve
- potentially high overhead of graphics API
- highly constrained memory layout & access model
- Need for many passes drives up bandwidth consumption

Motivation

11

GPGPU Programming to do $A + B$



Programming Model

CUDA: Unified Design

Advantage:

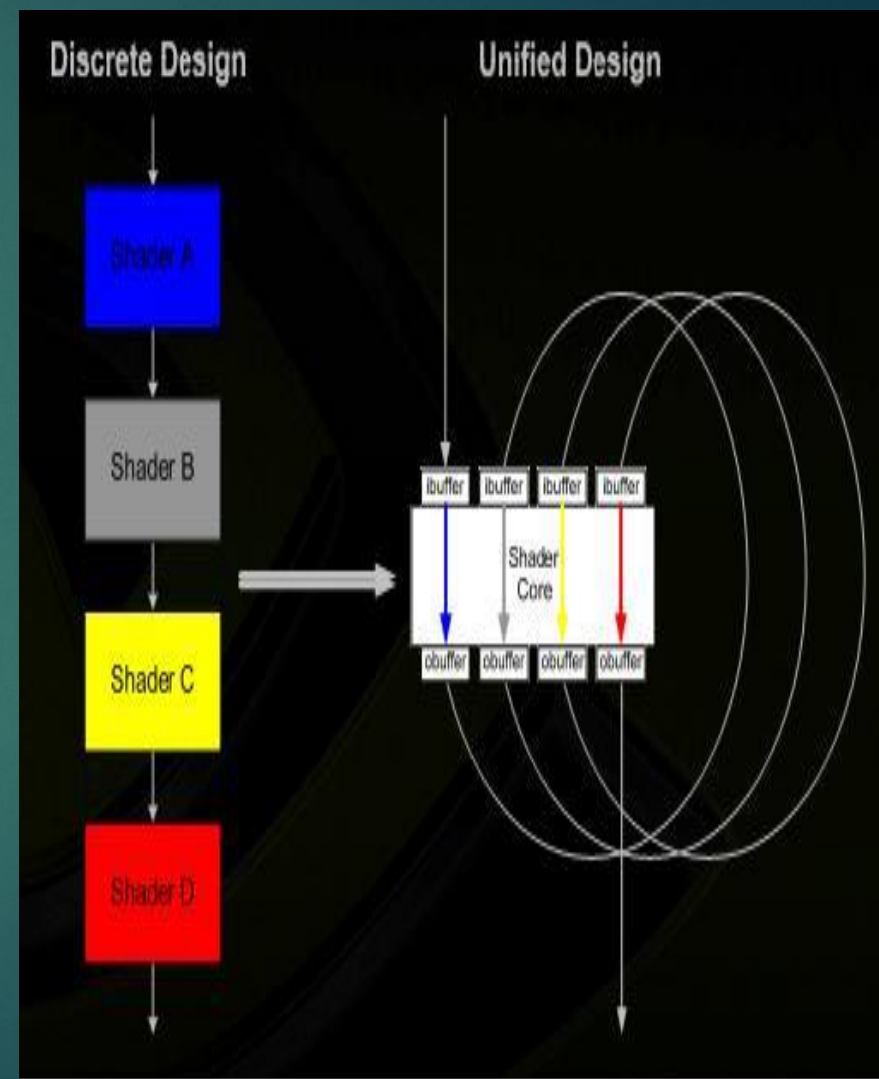
HW: fully generally data-parallel architecture.

- General thread launch
- Global load-store
- Parallel data cache
- Scalar architecture
- Integers, bit operation

SW: program the GPU in C

- Scalable data parallel execution/memory model
- C with minimal yet powerful extensions

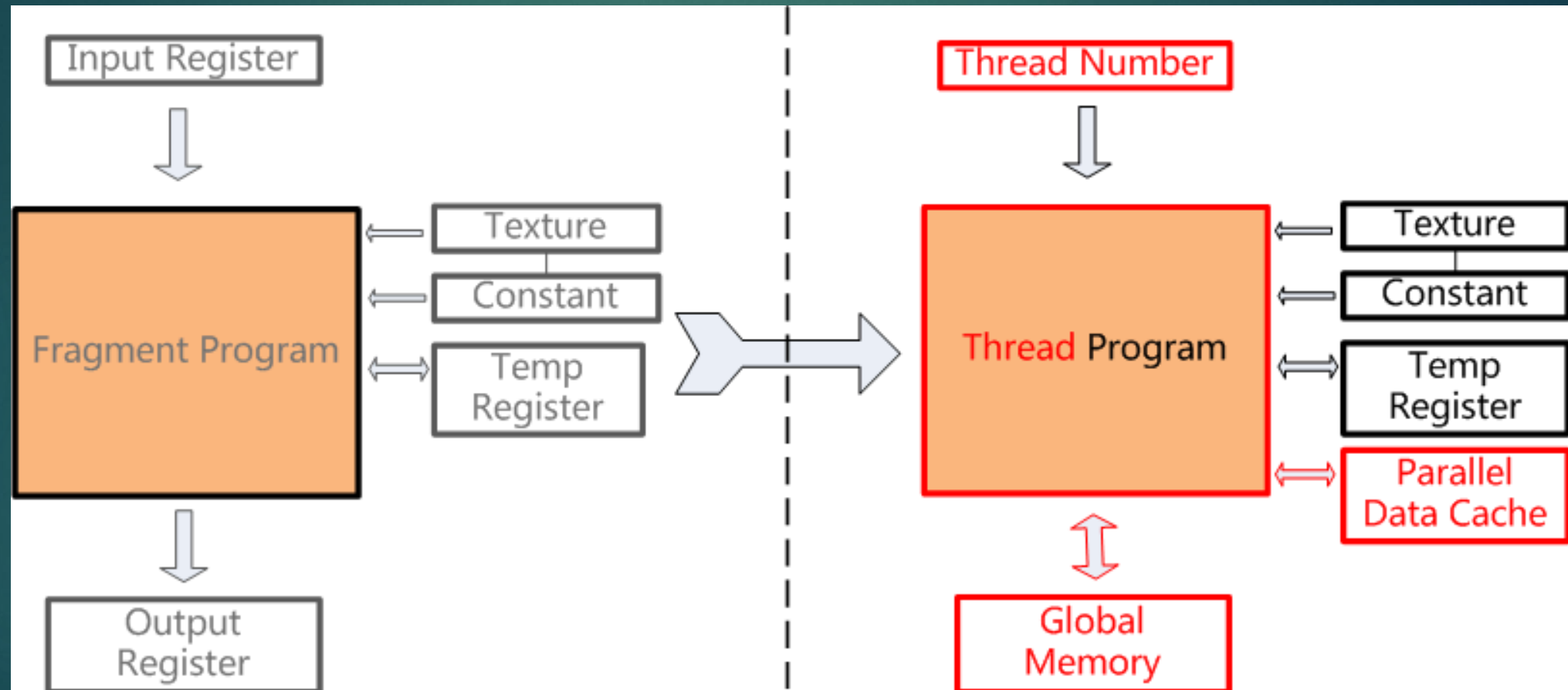
12



Motivation

13

From GPGPU to CUDA Programming Model

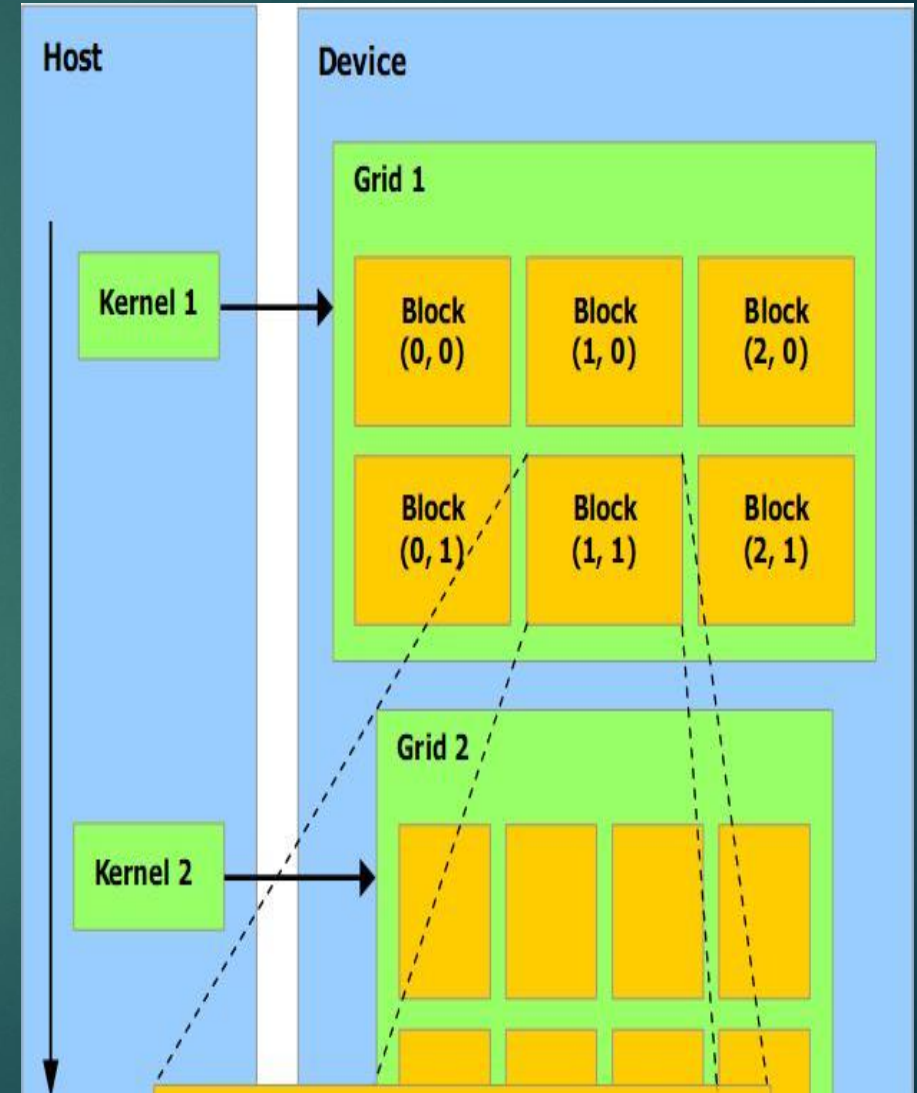


Programming Model

14

Important Concepts:

- Device: GPU, viewed as a co-processor.
- Host: CPU
- Kernel: data-parallel, computed-intensive positions of application running on the device.

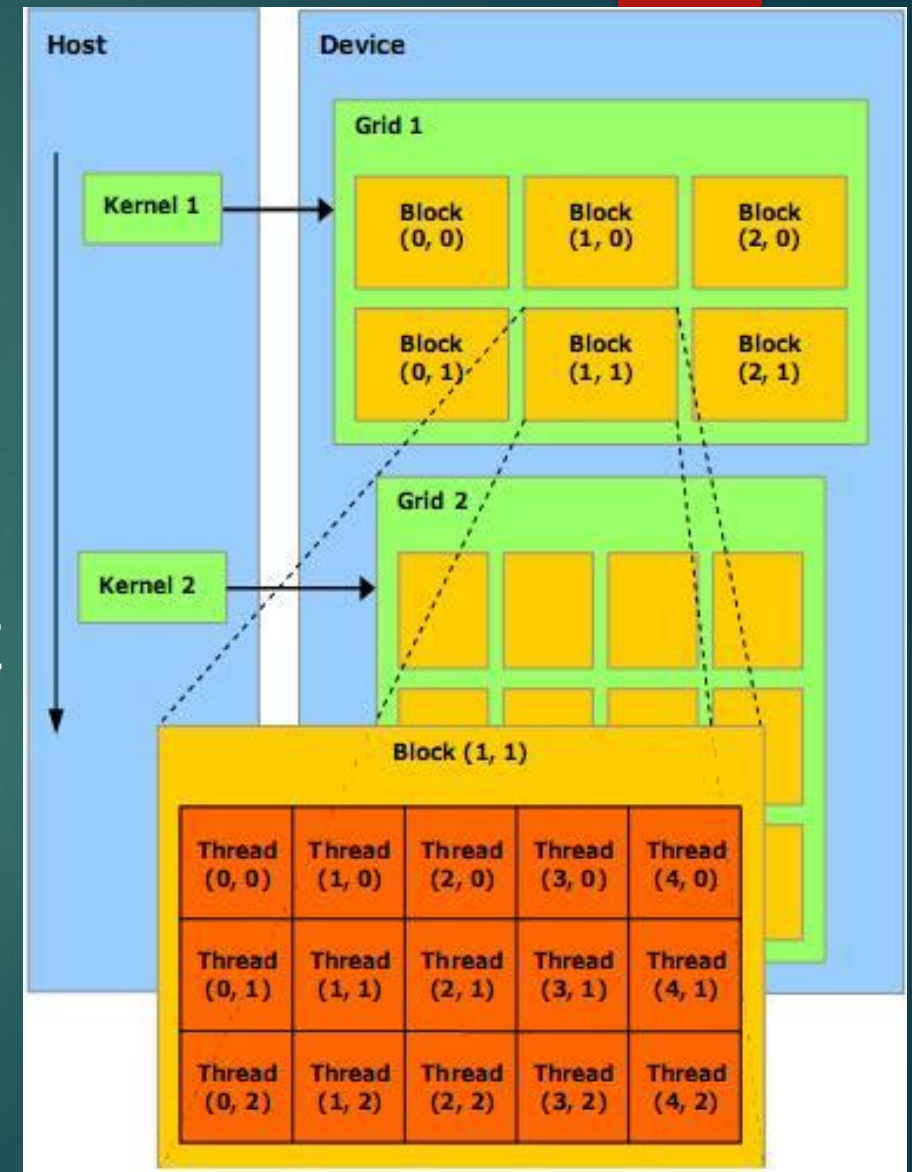


Programming Model

15

Important Concepts:

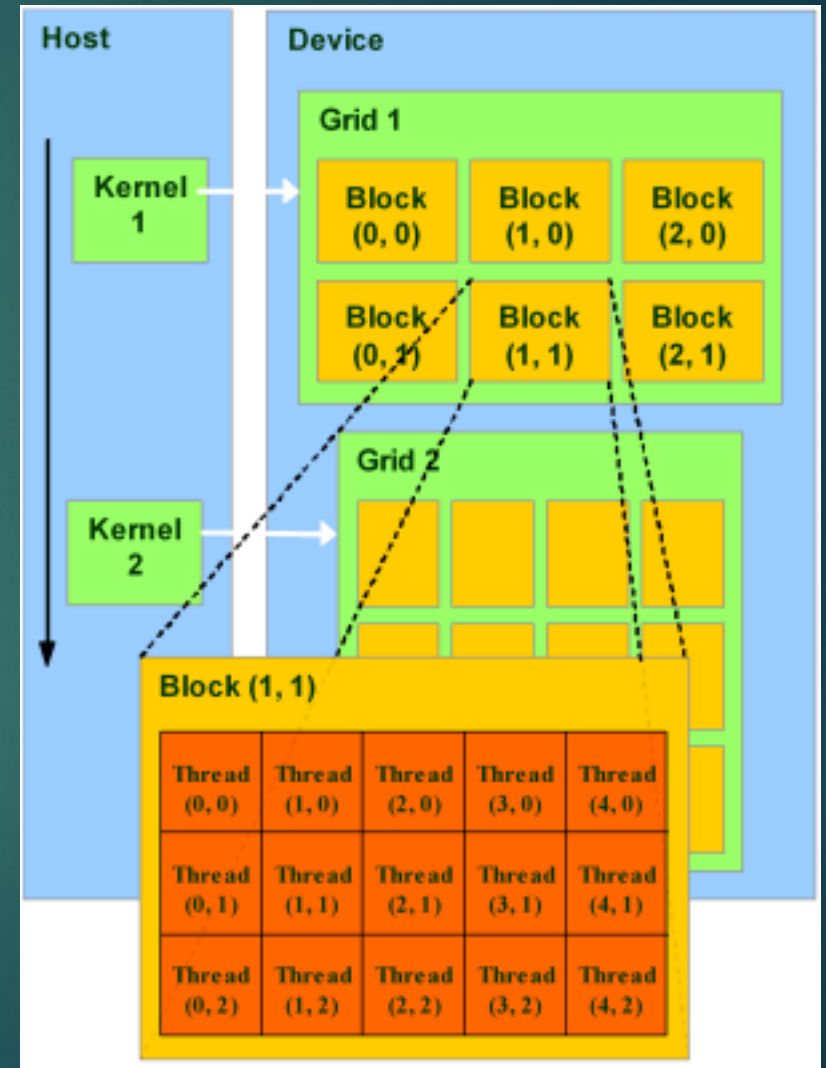
- Thread: basic execution unit
- Thread block: A batch of thread. Threads in a block cooperate together, efficiently share data. Thread/block have unique id
- Grid: A batch of thread block. that execute same kernel. Threads in different block in the same grid cannot directly communicate with each other



Programming Model CUDA Structure

16

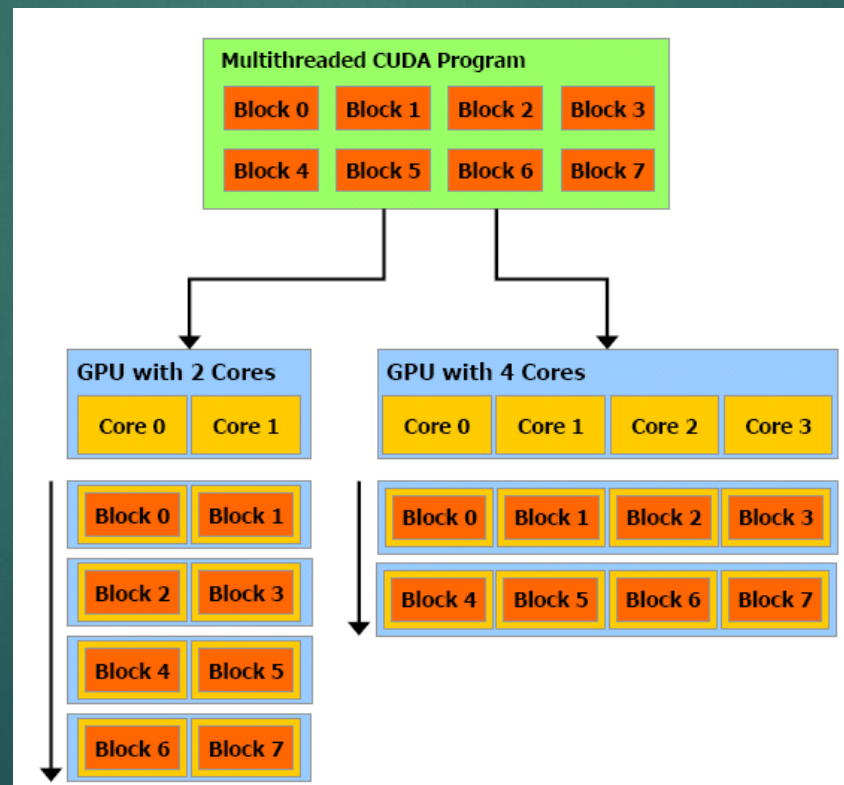
- ▶ Threads are grouped into thread blocks
- ▶ Blocks are grouped into a single grid
- ▶ The grid is executed on the GPU as a kernel



Programming Model Scalability

17

- ▶ Blocks map to cores on the GPU
- ▶ Allows for portability when changing hardware



Programming Model

Terms and Concepts

Each block and thread has a unique id within a block.

- ▶ threadIdx – identifier for a thread
- ▶ blockIdx – identifier for a block
- ▶ blockDim – size of the block

Unique thread id:

$(blockIdx * blockDim) + threadIdx$

Programming Model

19

Simple example (Matrix addition):

cpu c program:

```
void addVector (fload *a, float *b,
                fload *c, int N)
{
    int i, index,
    for (i = 0; i<N, i++) {
        c[index] = a[index] + b[index];
    }
}

void main()
{
    .....
    addVector(a, b, c, N);
    .....
}
```

cuda program:

```
__global__ void addVector (fload *a, float *b,
                           float *c)
{
    int i = threadIdx.x + blockDim.x*blockIdx.x;
    c[i] = a[i] + b[i];
}

Void main()
{
    ...
    // allocation & transfer data to GPU
    //Excuate on N/256 blocks of 256 threads each
    addVector << N/256, 256 >> ( d_A, d_B, d_C);
    ....
}
```

Device code

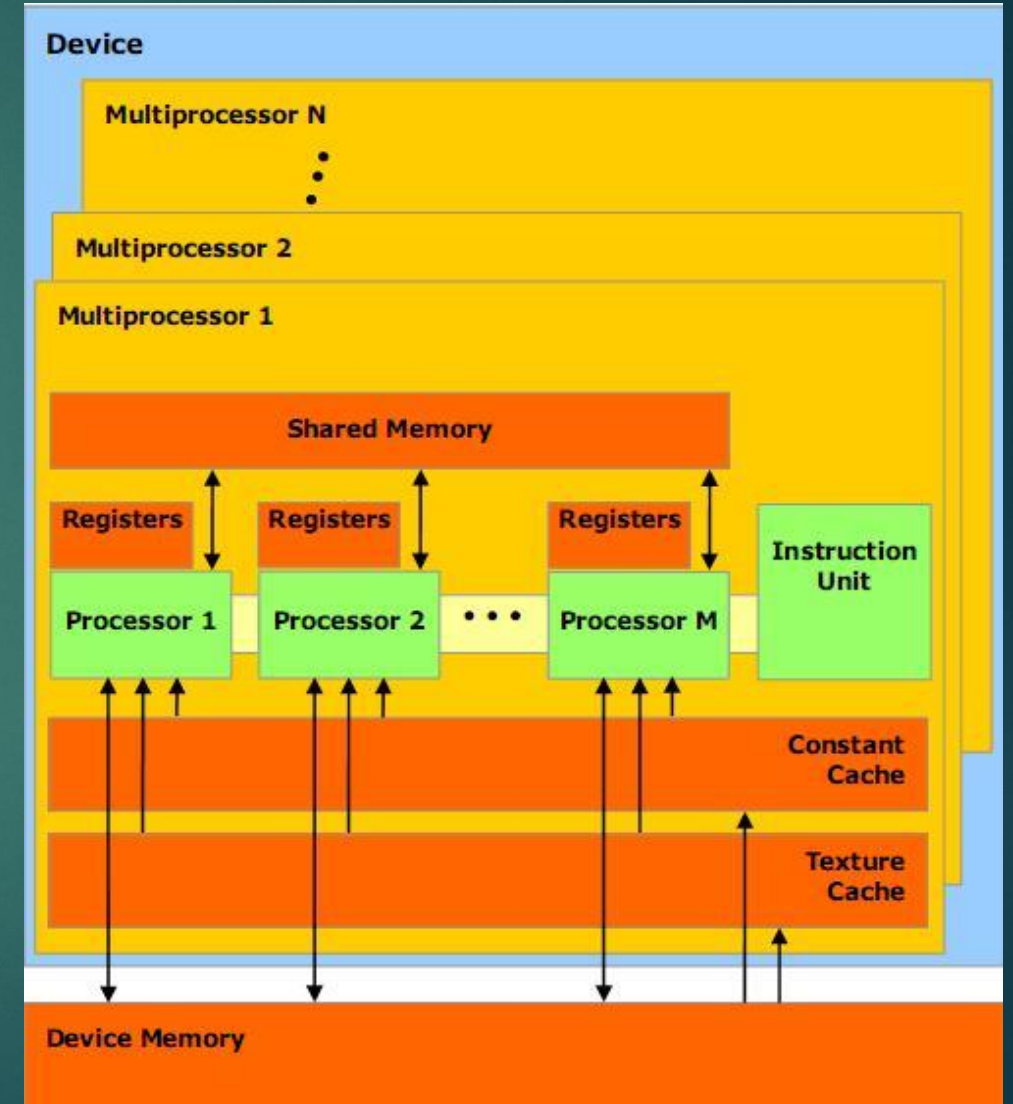
Host code

Programming Model

20

Hardware implementation:

A set of SIMD
Multiprocessors with On-Chip
shared memory

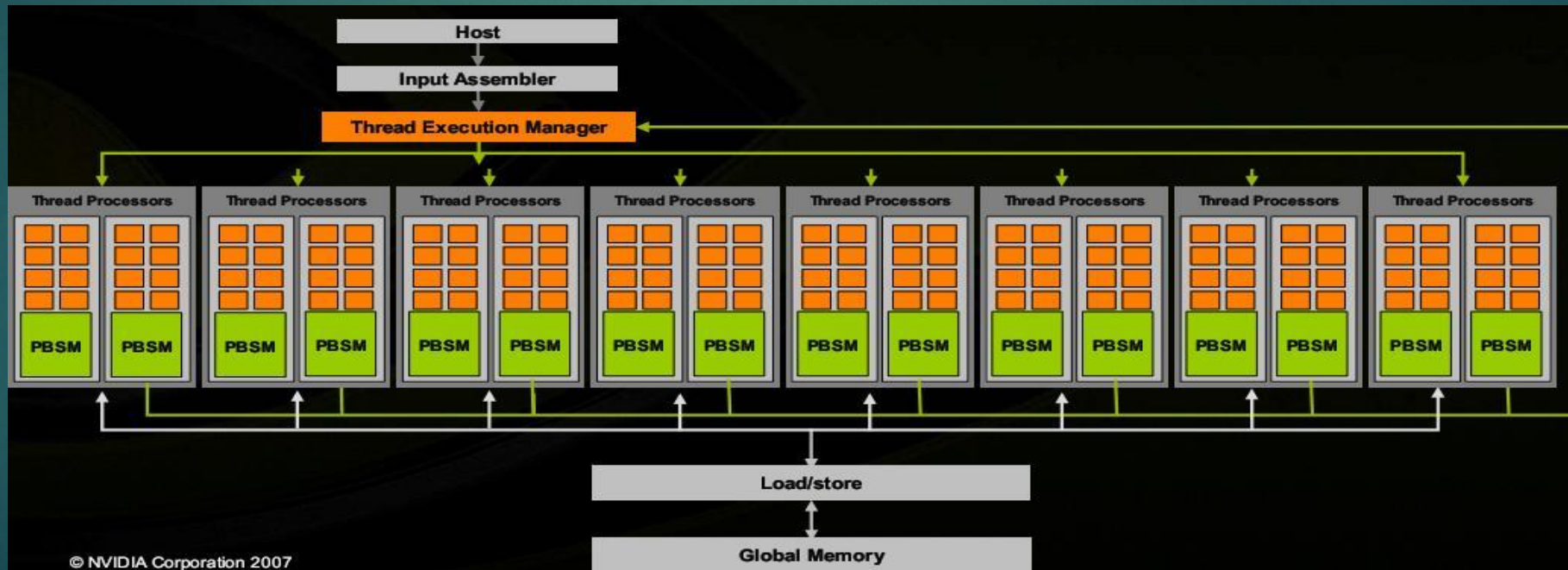


Programming Model

21

G80 Example:

- 16 Multiprocessors, 128 Thread Processors
- Up to 12,288 parallel threads active
- Per-block shared memory accelerates processing.

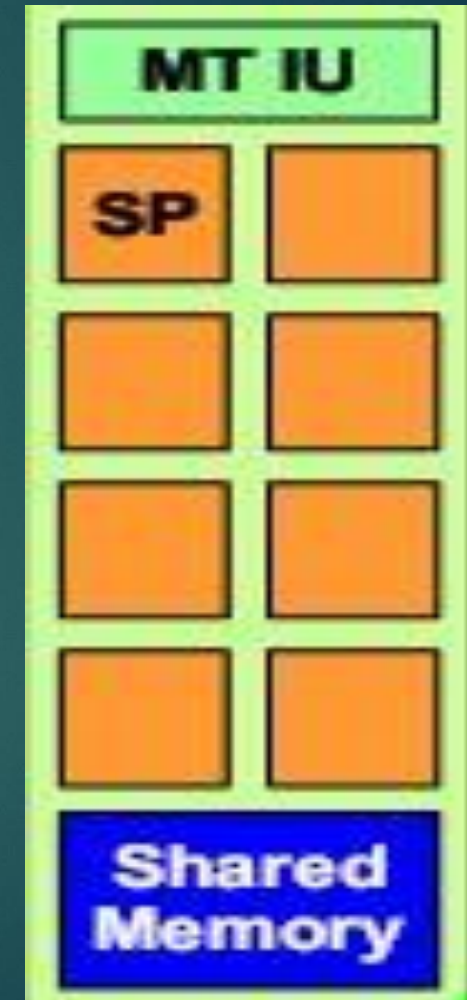


Programming Model

22

Streaming Multiprocessor (SM)

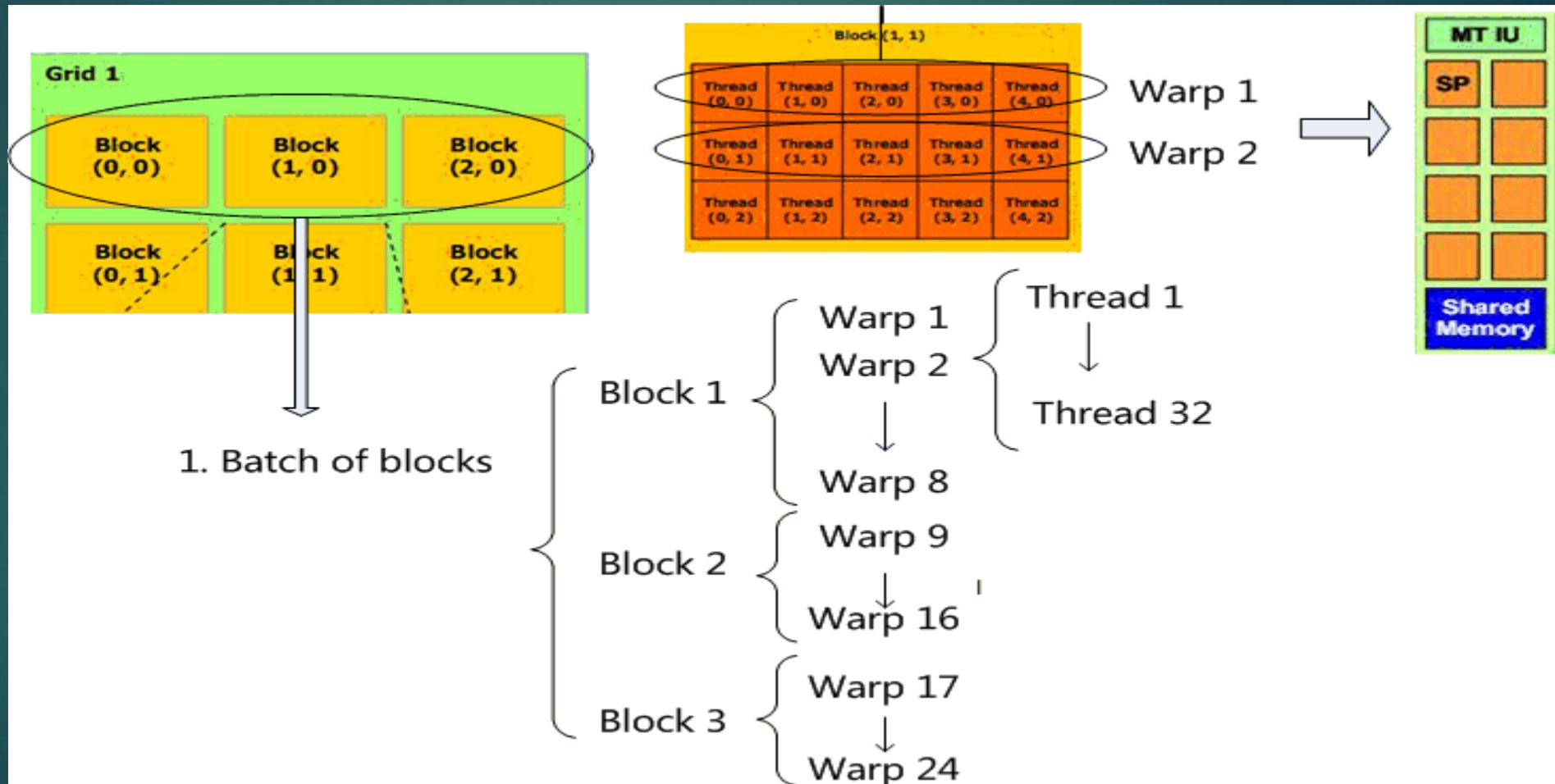
- Processing elements
 - 8 scalar thread processors
 - 32 GFLOPS peak at 1.35GHz
 - 8192 32-bit registers (32KB)
 - usual ops: float, int, branch...
- Hardware multithreading
 - up to 8 blocks (3 active) residents at once
 - up to 768 active threads in total
- 16KB on-chip memory
 - ▶ supports thread communication
 - ▶ shared amongst threads of a block



Programming Model

23

Execution Model:

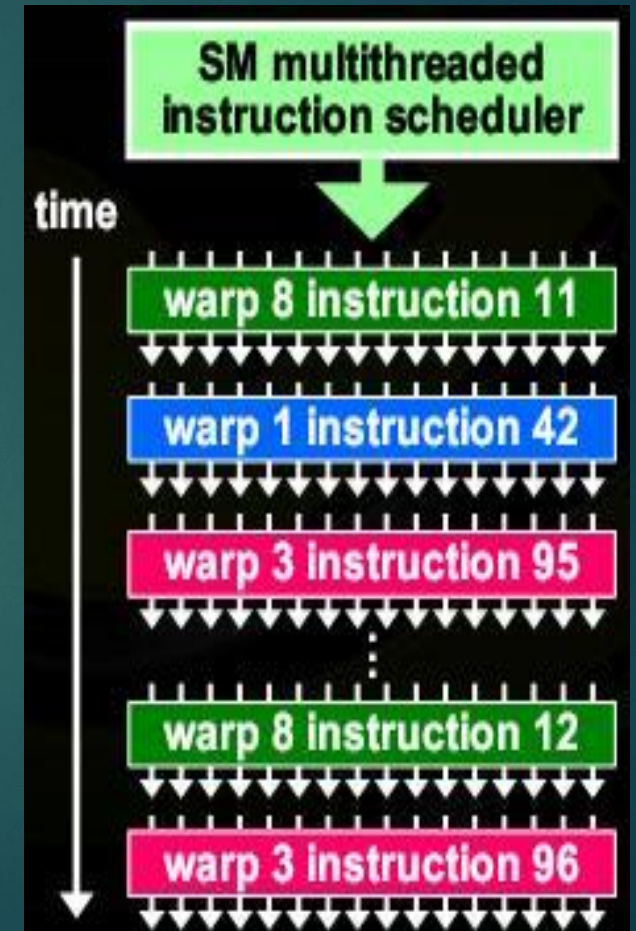


Programming Model

24

Single Instruction Multiple Thread (SIMT) Execution:

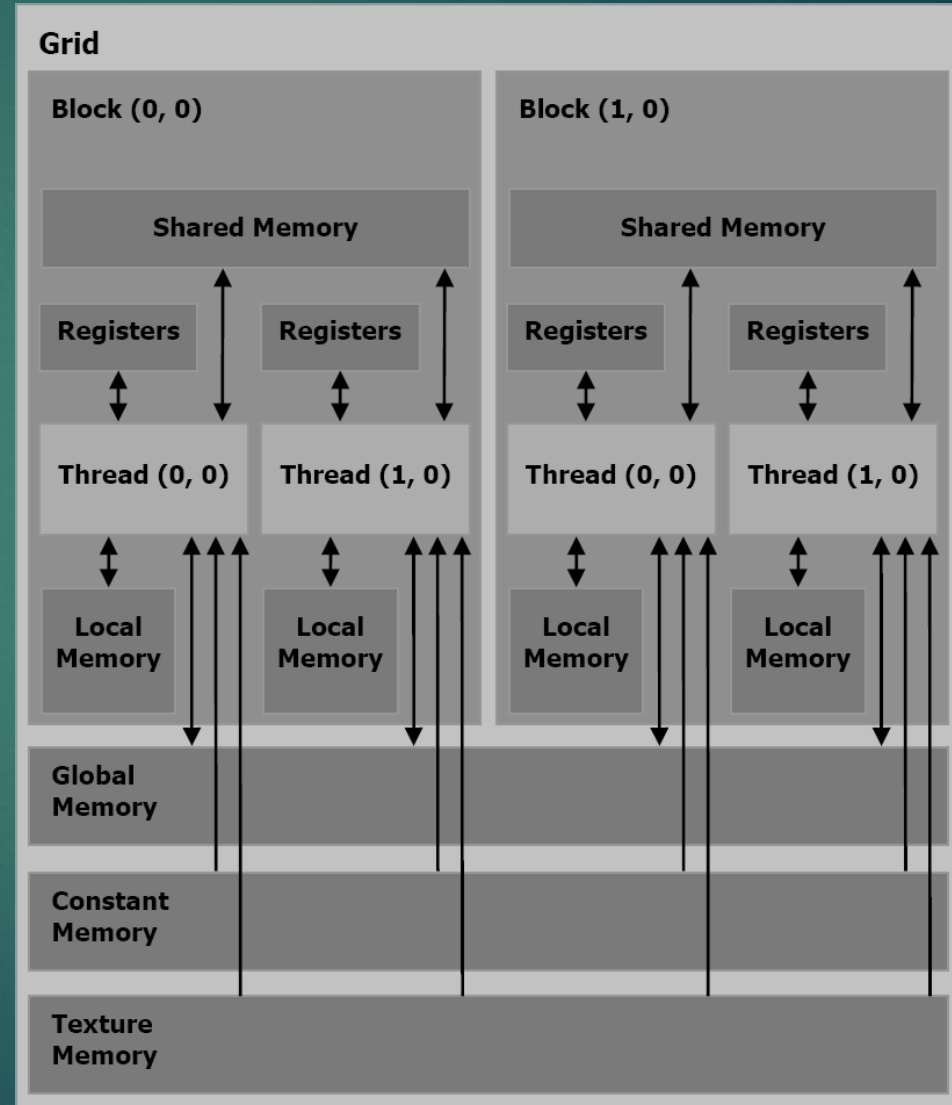
- Groups of 32 threads formed into **warps**
 - always executing same instruction
 - share instruction fetch/dispatch
 - some become **inactive** when code path diverges
 - hardware **automatically handles divergence**
- **Warps** are primitive unit of scheduling
 - ▶ pick 1 of 24 warps for each instruction slot.
 - ▶ all warps from all active blocks are time-sliced



Memory Model

25

There are 6 Memory Types :

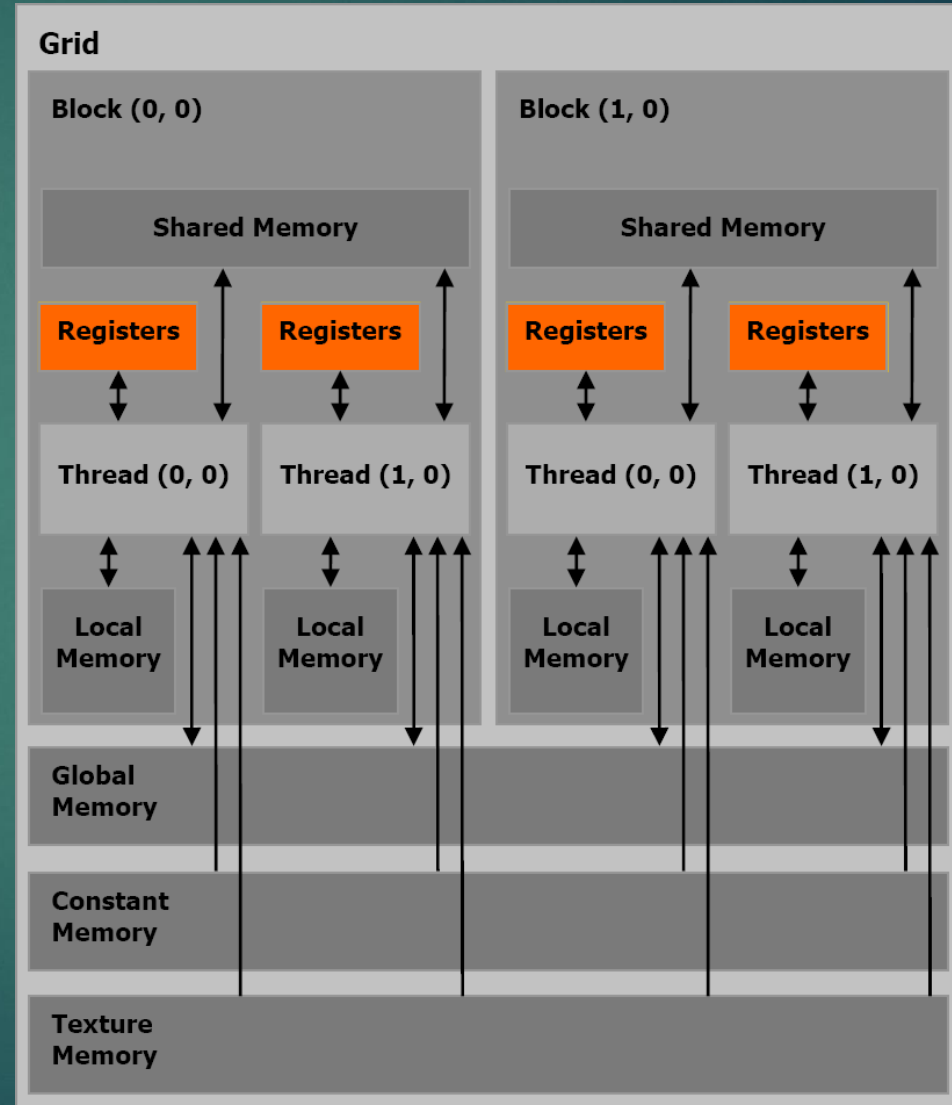


Memory Model

26

There are 6 Memory Types :

- **Registers**
 - on chip
 - fast access
 - per thread
 - limited amount
 - 32 bit

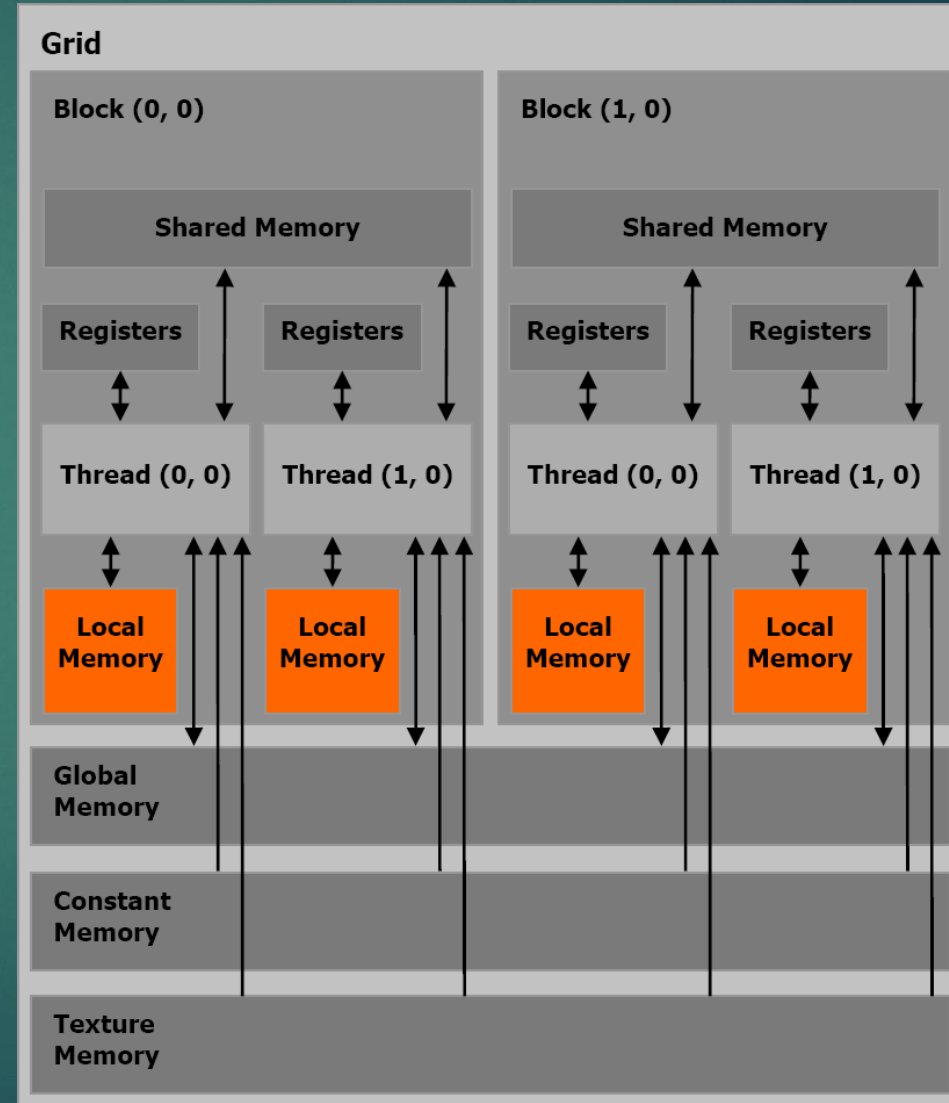


Memory Model

27

There are 6 Memory Types :

- Registers
- **Local Memory**
 - in DRAM
 - slow
 - non-cached
 - per thread
 - relative large

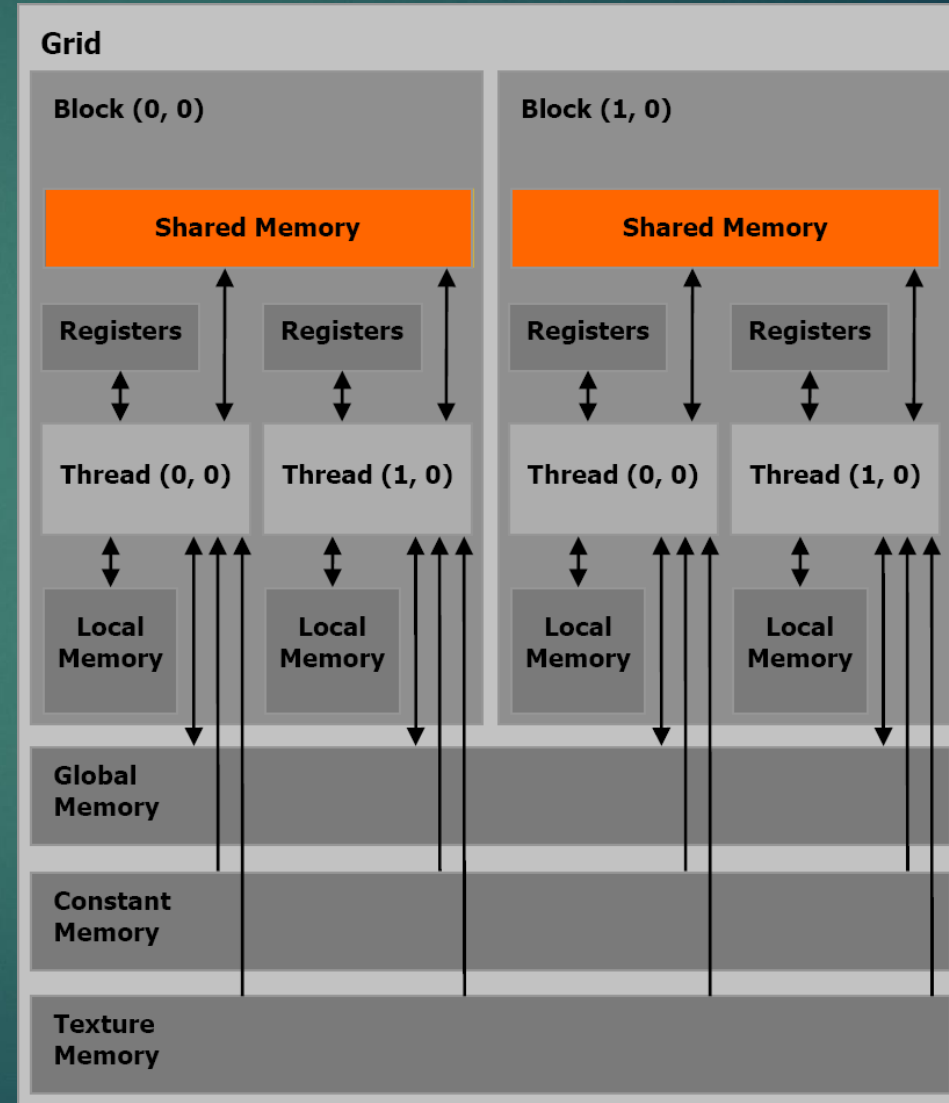


Memory Model

28

There are 6 Memory Types :

- Registers
- Local Memory
- **Shared Memory**
 - on chip
 - fast access
 - per block
 - 16 KByte
 - synchronize between threads

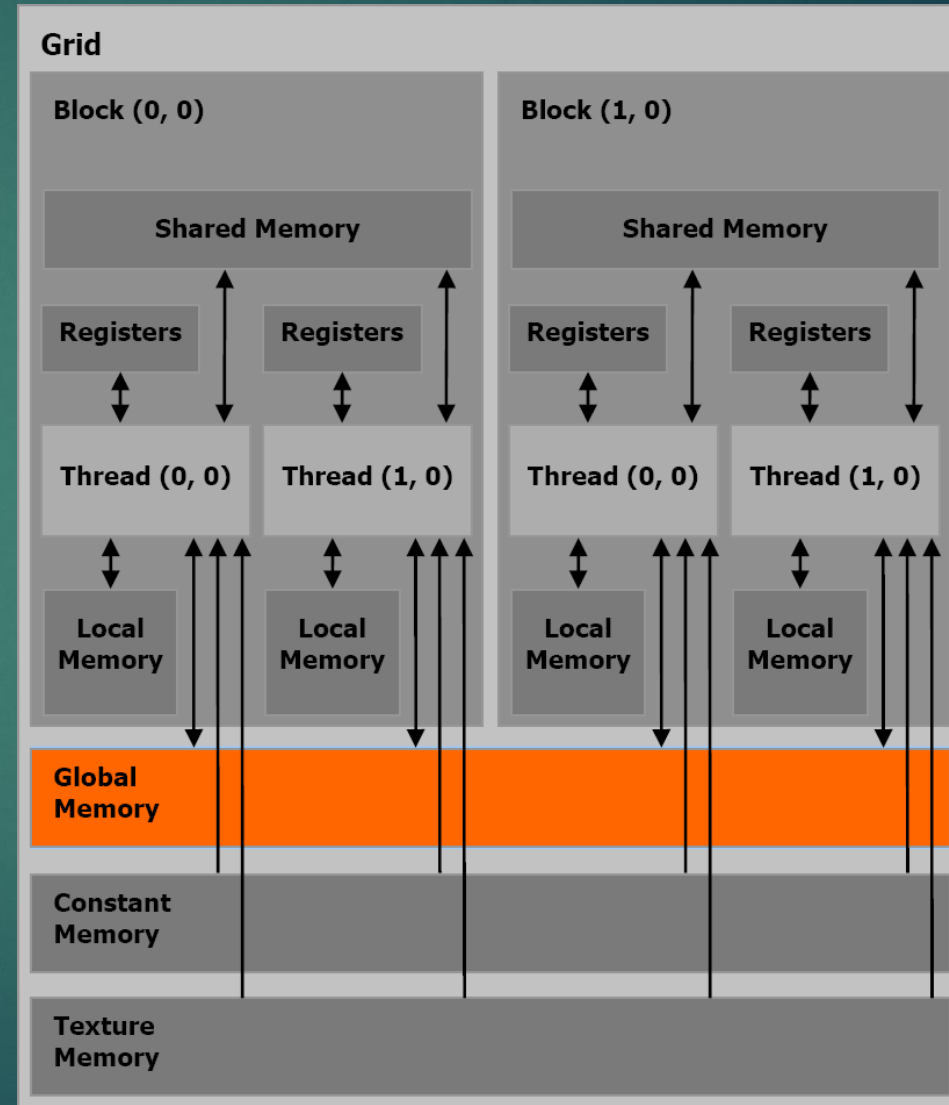


Memory Model

29

There are 6 Memory Types :

- Registers
- Local Memory
- Shared Memory
- **Global Memory**
 - in DRAM
 - slow
 - non-cached
 - per grid
 - communicate between grids

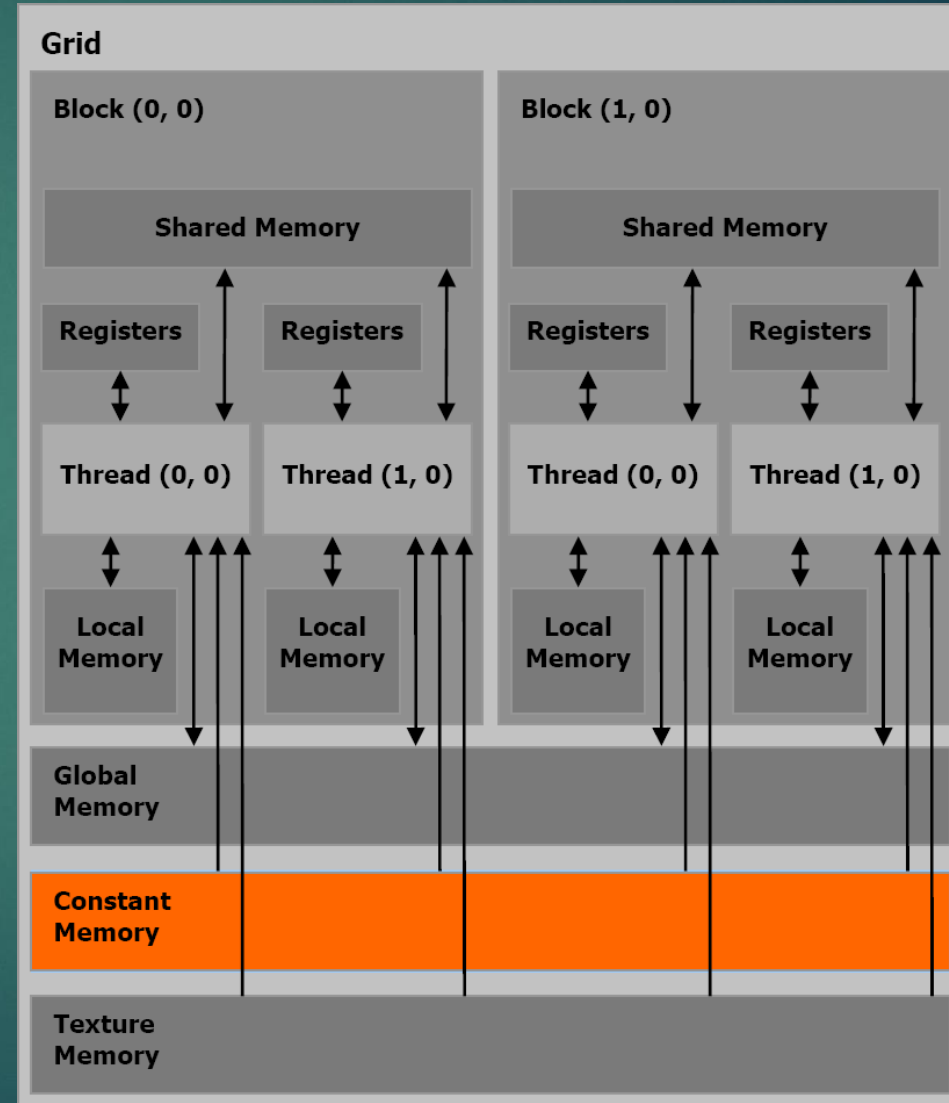


Memory Model

30

There are 6 Memory Types :

- Registers
- Local Memory
- Shared Memory
- Global Memory
- **Constant Memory**
 - in DRAM
 - cached
 - per grid
 - **read-only**

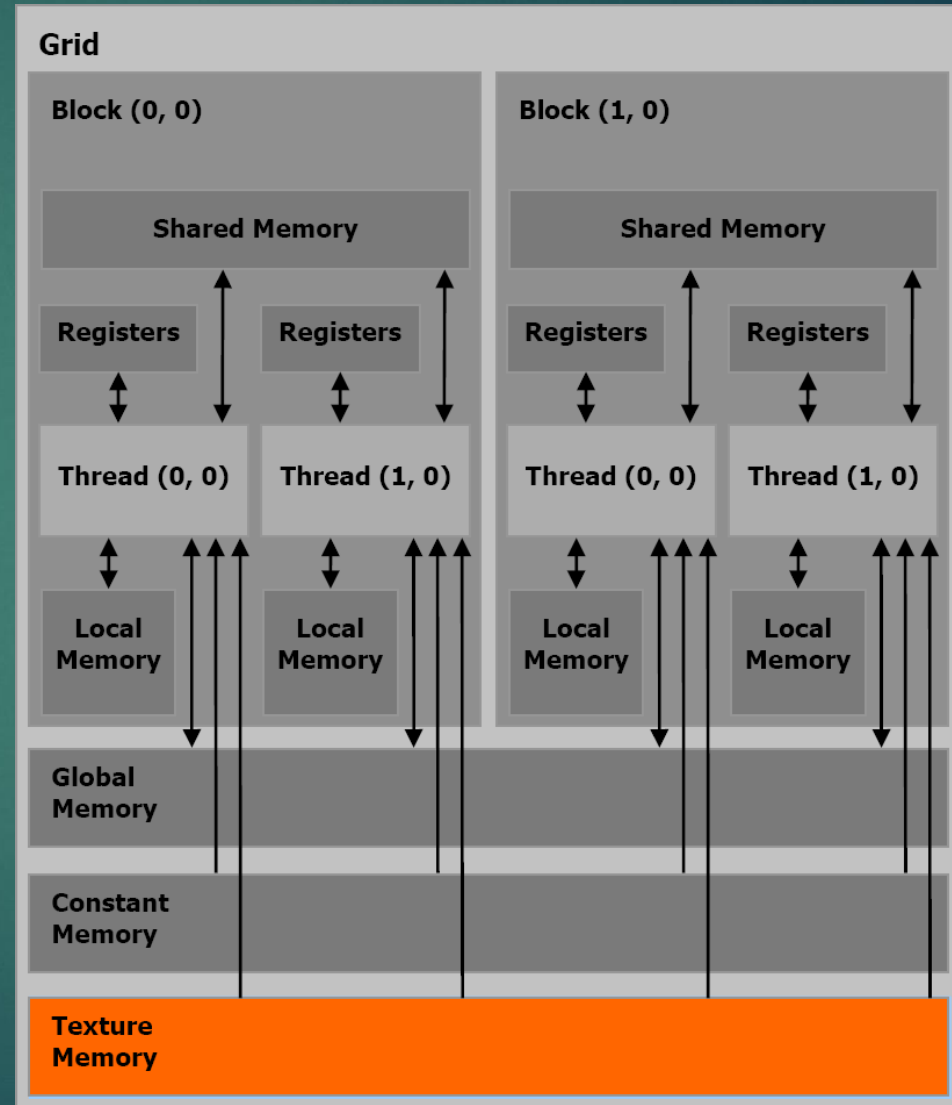


Memory Model

31

There are 6 Memory Types :

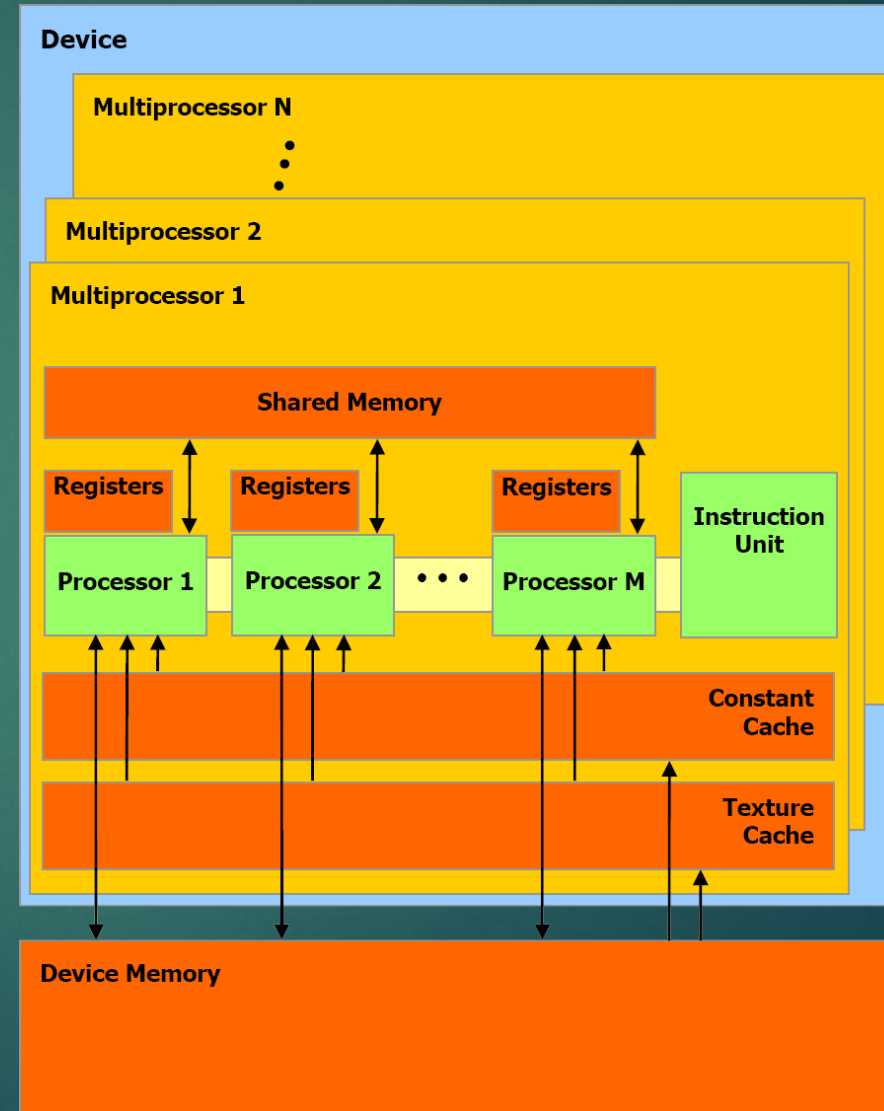
- Registers
- Local Memory
- Shared Memory
- Global Memory
- Constant Memory
- **Texture Memory**
 - in DRAM
 - cached
 - per grid
 - read-only



Memory Model

32

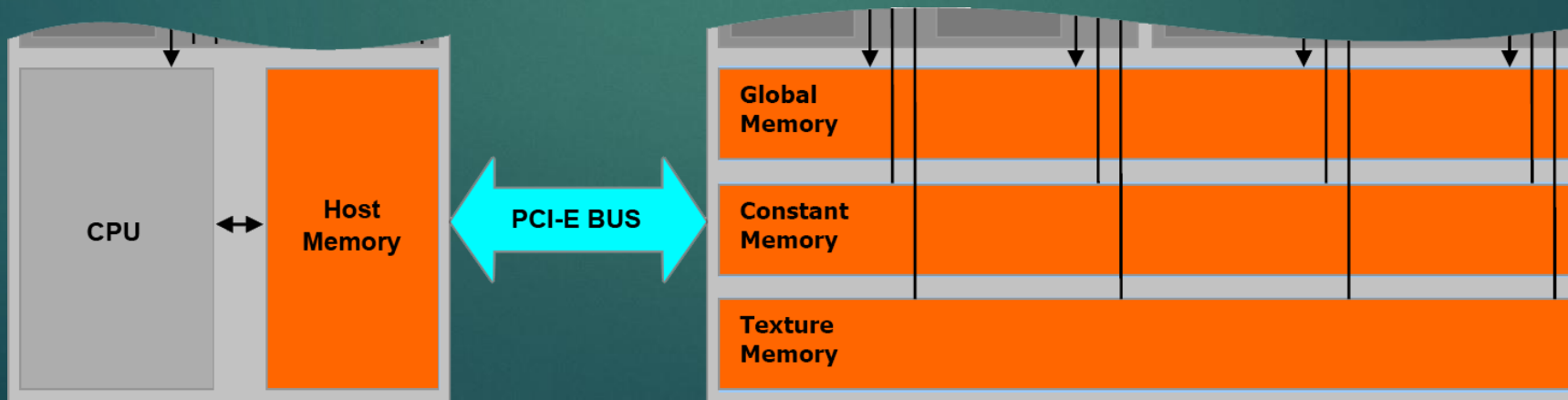
- Registers
- Shared Memory
 - on chip
- Local Memory
- Global Memory
- Constant Memory
- Texture Memory
 - in Device Memory



Memory Model

33

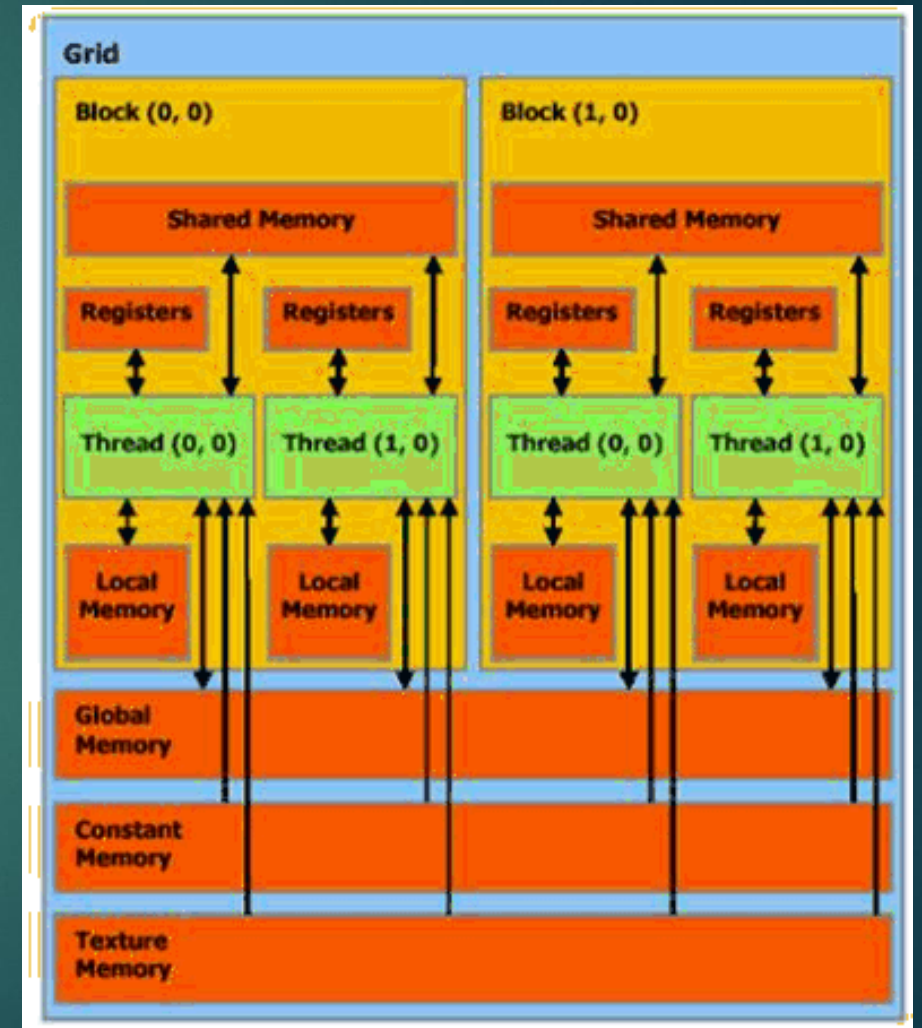
- Global Memory
- Constant Memory
- Texture Memory
 - managed by host code
 - persistent across kernels



Memory Hierarchy

34

- ▶ Shared memory much much faster than global
- ▶ Don't trust local memory
- ▶ Global, Constant, and Texture memory available to both host and cpu



CUDA API provides a easily path for users to write programs for GPU device .

It consists of:

- A minimal set of extensions to C/C++
 - type qualifiers
 - call-syntax
 - build-in variables
- A runtime library to support the execution
 - host component
 - device component
 - common component

CUDA C/C++ Extensions:

- New function type qualifiers

```
__host__    void HostFunc(...);    //executable on host  
__global__ void KernelFunc(...); //callable from host  
__device__ void DeviceFunc(...); //callable from device only
```

- Restrictions for device code (**__global__** / **__device__**)

- no recursive call
- no static variable
- no function pointer
- **__global__** function is asynchronous invoked
- **__global__** function must have `void` return type

CUDA API

Kernel Function Qualifiers

37

- ▶ `__device__`
- ▶ `__global__`
- ▶ `__host__`

Example in C:

CPU program

```
void increment_cpu(float *a, float b, int N)
```

CUDA program

```
__global__ void increment_gpu(float *a, float b, int N)
```

CUDA C/C++ Extensions:

- New variable type qualifiers

```
__device__ int GlobalVar; //in global memory, lifetime of app  
__const__ int ConstVar; //in constant memory, lifetime of app  
__shared__ int SharedVar; //in shared memory, lifetime of blocks
```

- Restrictions

- no external usage
- only file scope
- no combination with `struct` or `union`
- no initialization for __shared__

CUDA API

Variable Type Qualifiers

- ▶ Specify how a variable is stored in memory
- ▶ `__device__`
- ▶ `__shared__`
- ▶ `__constant__`

Example:

```
__global__ void increment_gpu(float *a, float b, int N)
{
    __shared__ float shared[];
}
```

CUDA API

40

CUDA C/C++ Extensions:

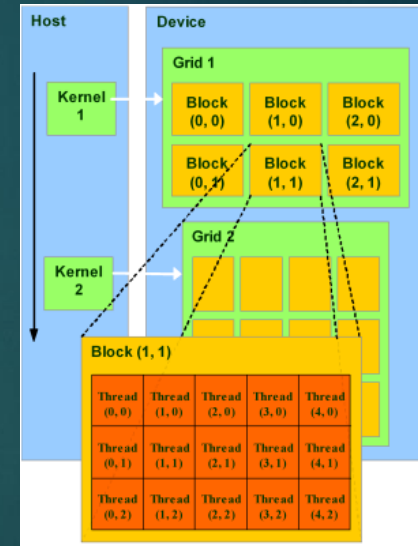
- New syntax to invoke the device code

KernelFunc<<< **Dg, Db, Ns, S** >>> (...);

- Dg: dimension of grid
- Db: dimension of block
- Ns: optional, shared memory for external variables
- S : optional, associated stream

- New build-in variables for indexing the threads

- gridDim: dimension of the whole grid
- blockIdx: index of the current block
- blockDim: dimension of each block in the grid
- threadIdx: index of the current thread



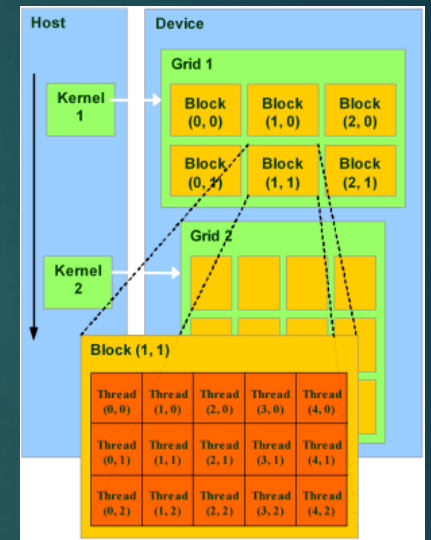
CUDA API

Calling the Kernel

41

- ▶ Calling a kernel function is much different from calling a regular function

```
Void main(){  
  Int blocks = 256;  
  Int threadsperblock = 512;  
  mycudafunc<<<blocks,threadsperblock>>>(some parameter);  
}
```



CUDA API

GPU Memory Allocation / Release

Host (CPU) manages GPU memory:

- ▶ `cudaMalloc (void ** pointer, size_t nbytes)`
- ▶ `cudaMemset (void * pointer, int value, size_t count);`
- ▶ `cudaFree (void* pointer)`

```
void main(){  
    int n = 1024;  
    int nbytes = 1024*sizeof(int);  
    int * d_a = 0;  
    cudaMalloc( (void**)&d_a, nbytes );  
    cudaMemset( d_a, 0, nbytes);  
    cudaFree(d_a);  
}
```

CUDA API

Memory Transfer

```
cudaMemcpy( void *dst, void *src, size_t nbytes,  
            enum cudaMemcpyKind direction);
```

- ▶ returns after the copy is complete blocks CPU
- ▶ thread doesn't start copying until previous CUDA calls complete
- ▶ enum cudaMemcpyKind
 - ▶ cudaMemcpyHostToDevice
 - ▶ cudaMemcpyDeviceToHost
 - ▶ cudaMemcpyDeviceToDevice

CUDA API

Host Synchronization

44

- ▶ All kernel launches are asynchronous
 - ▶ control returns to CPU immediately
 - ▶ kernel starts executing once all previous CUDA calls have completed
- ▶ Memcopies are synchronous
 - ▶ control returns to CPU once the copy is complete
 - ▶ copy starts once all previous CUDA calls have completed
- ▶ `cudaThreadSynchronize()`
 - ▶ blocks until all previous CUDA calls complete
- ▶ Asynchronous CUDA calls provide:
 - ▶ non-blocking memcopies
 - ▶ ability to overlap memcopies and kernel execution

CUDA Runtime Library:

- Common component
 - Vector/Texture Types
 - Mathematical/Time Functions
- Device component
 - Mathematical/Time/Texture Functions
 - Synchronization Function
 - `__syncthreads()`
 - Type Conversion/Casting Functions

CUDA API

46

CUDA Runtime Library:

- Host component
 - Structure
 - Driver API
 - Runtime API
 - Functions
 - Device, Context, Memory, Module, Texture management
 - Execution control
 - Interoperability with OpenGL and Direct3D

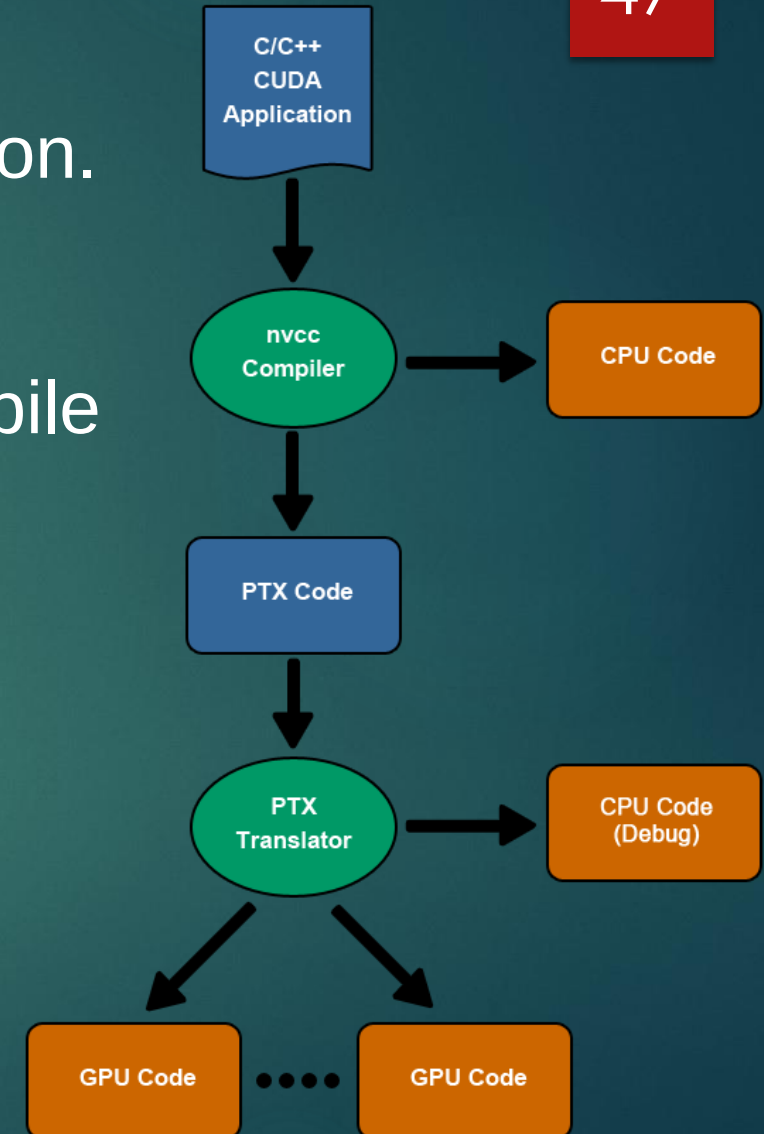
CUDA API

47

The CUDA source file uses `.cu` as extension. It contains host and device source codes.

The CUDA Compiler Driver `nvcc` can compile it and generate CPU/PTX binary code. (PTX: Parallel Thread Execution, a device independent VM code)

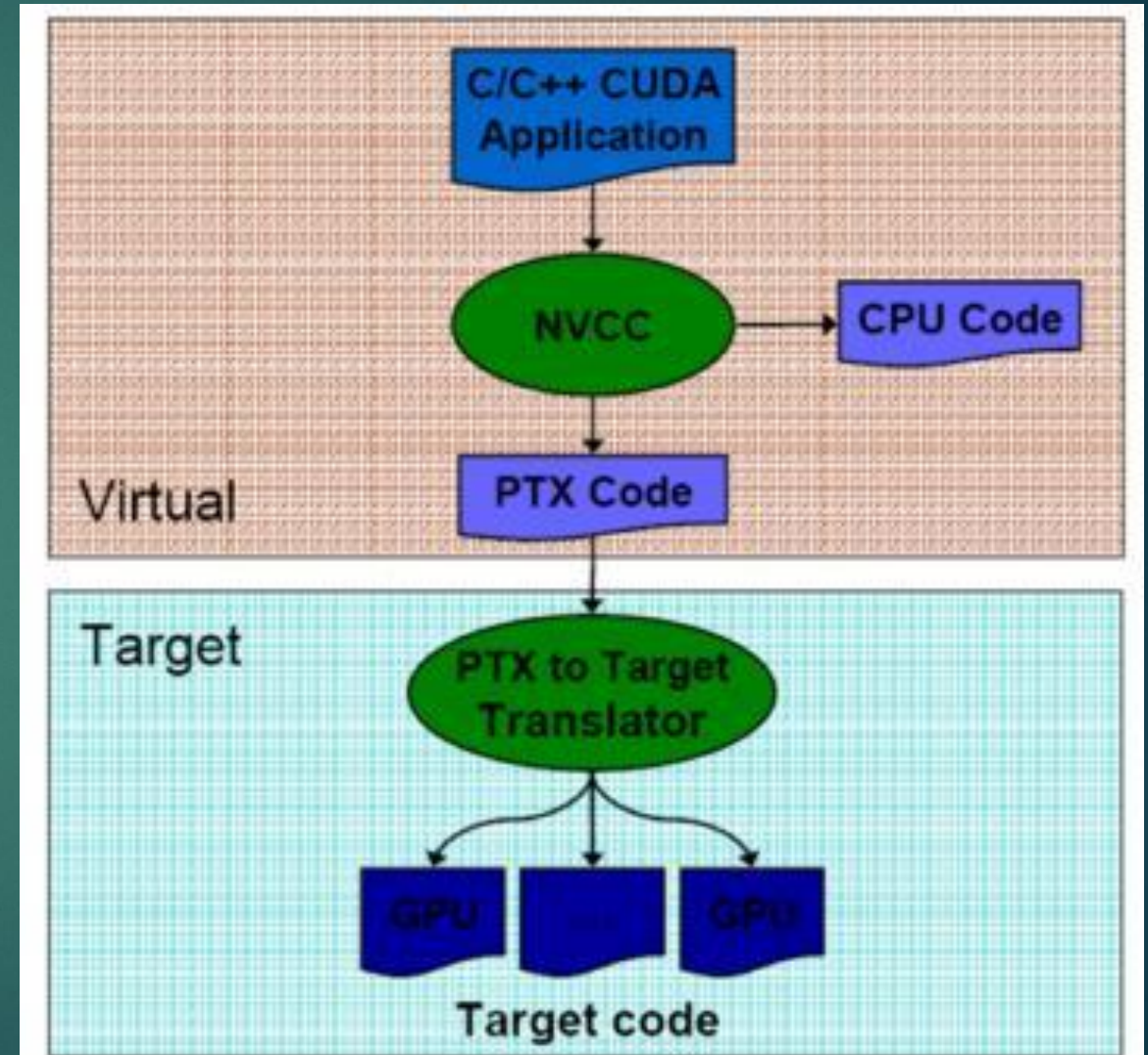
PTX code may be further translated for special GPU-Arch.



CUDA API

NVCC compiler

- Compiles C or PTX code (CUDA instruction set architecture)
- Compiles to either PTX code or binary (cu bin object)



Development: Basic Idea

49

1. Allocate equal size of memory for both host and device
2. Transfer data from host to device
3. Execute kernel to compute on data
4. Transfer data back to host

Programming Model

50

Simple example (Matrix addition):

cpu c program:

```
void addVector (float *a, float *b,
               float *c, int N)
{
    int i, index,
    for (i = 0; i < N, i++) {
        c[index] = a[index] + b[index];
    }
}

void main()
{
    .....
    addVector(a, b, c, N);
    .....
}
```

cuda program:

```
__global__ void addVector (float *a, float *b,
                          float *c)
{
    int i = threadIdx.x + blockDim.x*blockIdx.x;
    c[i] = a[i] + b[i];
}

Void main()
{
    ...
    // allocation & transfer data to GPU
    //Excuate on N/256 blocks of 256 threads each
    addVector << N/256, 256 >> ( d_A, d_B, d_C);
    ....
}
```

Device code

Host code

Example Program

51

```
__global__ void arradd (float *a, float f, int N)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) a[i] = a[i] + float;
}
```

GPU

```
int main()
{
    float h_a[N];
    float *d_a;
    cudaMalloc ((void **) &a_d, SIZE);

    cudaThreadSynchronize ();
    cudaMemcpy (d_a, h_a, SIZE, cudaMemcpyHostToDevice));

    arradd <<< n_blocks, block_size >>> (d_a, 10.0, N);

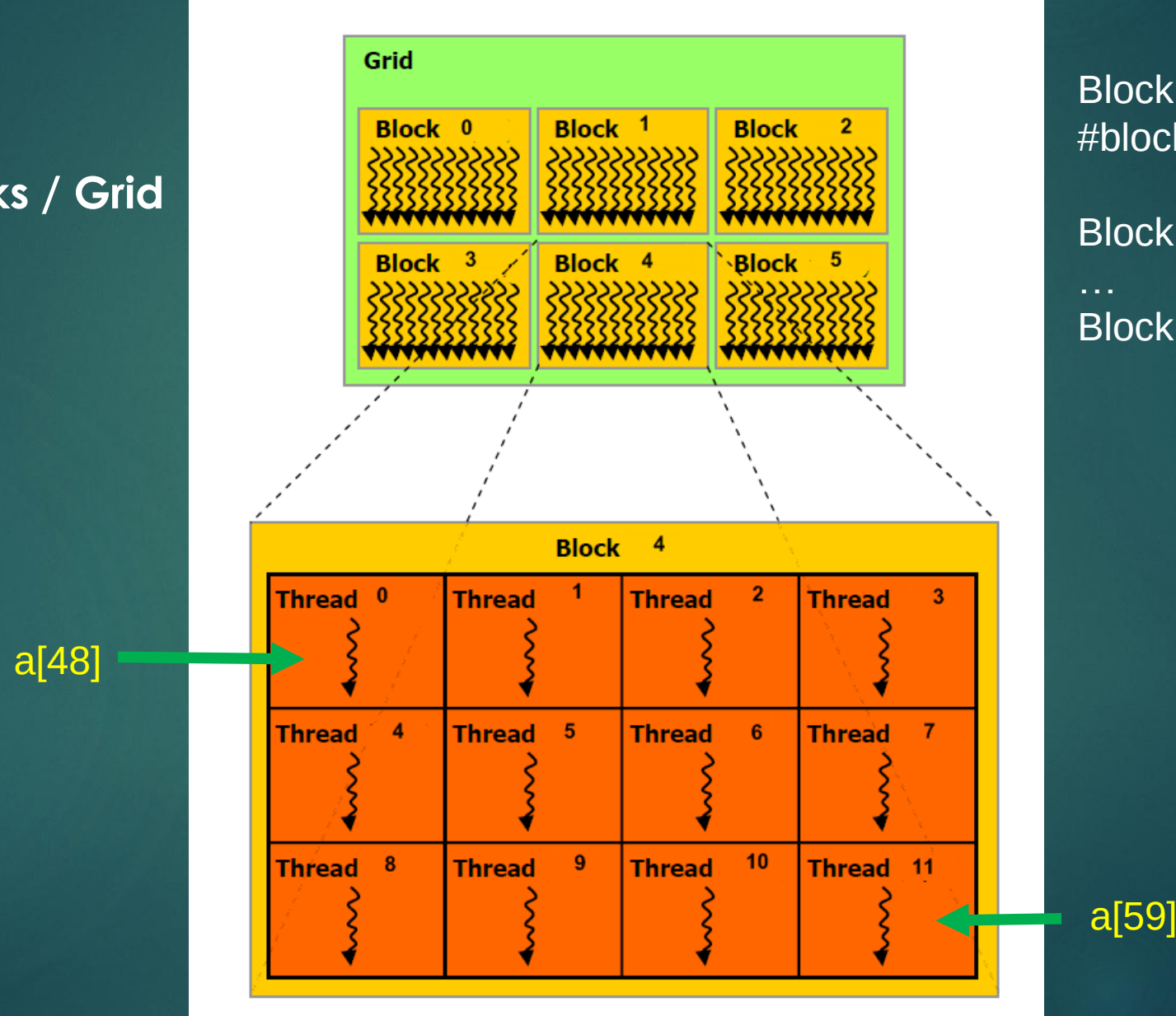
    cudaThreadSynchronize ();
    cudaMemcpy (h_a, d_a, SIZE, cudaMemcpyDeviceToHost));
    CUDA_SAFE_CALL (cudaFree (a_d));
}
```

CPU

Example Program: Structure

52

► Threads / Blocks / Grid



Pro & Con

53

CUDA allows

- massive parallel computing
- with a relative low price
- high integrated solution
- personal supercomputing
- ecofriendly production
- easy to learn

Pro & Con

54

Problem

- slightly low precision
- limited support for IEEE-754
- no recursive function call
- hard to use for irregular join/fork logic
- no concurrency between jobs

55

-
- The diagram illustrates a multi-processor system architecture. At the top, three processing units are shown: SM-0, SM-1, and SM-N-1. Each unit contains a 'Registers' block (256 KB per SM in A100), an 'L1/SWEM' block (192 KB in A100), and a 'Read only' block. Arrows indicate data flow from the L1/SWEM and Read only blocks to a shared 'L2 Cache (40 MB in A100)' block. Below the L2 Cache is the 'Global Memory (DRAM, 40 GB in A100)' block. Arrows show the connection between the L2 Cache and the Global Memory.

GPU Computing Applications						
Libraries and Middleware						
cuDNN TensorRT	cuFFT cuBLAS cuRAND cuSPARSE	CUDA MAGMA	Thrust NPP	VSPIPL SVM OpenCurrent	PhysX OptiX iRay	MATLAB Mathematica
Programming Languages						
C	C++	Fortran	Java Python Wrappers	DirectCompute	Directives (e.g. OpenACC)	
			CUDA-Enabled NVIDIA GPUs			
NVIDIA Ampere Architecture (compute capabilities 8.x)					Tesla A Series	
NVIDIA Turing Architecture (compute capabilities 7.x)		GeForce 2000 Series	Quadro RTX Series	Tesla T Series		
NVIDIA Volta Architecture (compute capabilities 7.x)	DRIVE/JETSON AGX Xavier		Quadro GV Series	Tesla V Series		
NVIDIA Pascal Architecture (compute capabilities 6.x)	Tegra X2	GeForce 1000 Series	Quadro P Series	Tesla P Series		
	 Embedded	 Consumer Desktop/Laptop	 Professional Workstation	 Data Center		

END

QUESTIONS & ANSWERS